

École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Rapport de stage

Évaluation de méthodes d'adaptation de domaine pour la prédiction
à partir de séries temporelles soumises à un changement de distribution

Stage de deuxième année

Élève : Arnaud GRASSIAN

Tuteur de stage de l'organisme d'accueil : Mounir ATIQ

Enseignante référente : Mathilde MOUGEOT

Établissement d'accueil : ENSIIE, 1 place de la Résistance, 91000 Évry-Courcouronnes

Fonction de l'encadrant : enseignant-chercheur

Période du stage : 1er juillet 2026 — 25 août 2026

Préambule

Résumé

Ce rapport présente le travail réalisé durant mon stage de deuxième année, effectué à l'ENSIIE sous l'encadrement de Mounir Atiq, enseignant-chercheur et l'un des auteurs de la bibliothèque ADAPT.

La plupart des modèles d'apprentissage supposent que les données d'entraînement et les données d'usage proviennent d'une même distribution. Cette hypothèse est rarement vérifiée sur des séries temporelles réelles, où les conditions de mesure, les capteurs ou les populations observées changent au cours du temps. L'adaptation de domaine regroupe les méthodes qui cherchent à corriger cet écart. La bibliothèque Python ADAPT en implémente une trentaine, mais leur évaluation systématique sur des séries temporelles restait limitée.

L'objectif du stage était de construire cette évaluation, j'ai procédé en deux temps : d'abord sur des séries synthétiques où je contrôlais exactement le type et l'intensité du décalage, puis sur 2 jeux de données réels couvrant 2 tâches différentes (prévision et classification).

Sur les séries synthétiques, seuls les décalages qui changent la façon dont la série évolue coûtent quelque chose. Ajouter une constante, une pente, ou modifier l'amplitude du cycle ne dégrade pas la prévision, parce que le modèle absorbe ces transformations. Sur les 9 méthodes testées, 3 seulement améliorent le transfert, et 3 le dégradent fortement.

Sur la prévision de température, un simple recentrage par ville fait mieux que 2 des 3 méthodes retenues, sans utiliser la moindre donnée annotée de la ville cible.

Sur la classification d'activité, le taux de bonnes prédictions atteint 0,899 sans adaptation, mais ce chiffre global cache des écarts importants selon la personne et l'activité.

Mots clés

Adaptation de domaine, séries temporelles, changement de distribution, transfert, prévision, classification, décomposition saisonnière, ADAPT.

Remerciements

Je remercie d'abord Mounir Atiq et Mathilde Mougeot d'avoir rendu ce stage possible. Mon calendrier était serré et l'édition de la convention a demandé de la réactivité de leur part comme de celle de l'administration de l'école.

Je remercie Mounir Atiq pour son encadrement tout au long de ces deux mois. Nos rencontres régulières ont orienté le travail à chaque étape, et plusieurs des résultats présentés ici viennent directement de questions qu'il a posées et auxquelles je n'aurais pas pensé seul.

Je remercie Mohamed Kassoul, avec qui j'ai partagé ces deux mois de stage, pour les échanges techniques quotidiens, l'entraide sur les outils communs et la bonne ambiance de travail.

Je remercie également Mathilde Mougeot, enseignante référente de ce stage, pour son suivi et sa disponibilité dans les démarches administratives.

Apports personnels

Ce stage m'a fait découvrir le travail de recherche appliquée, très différent du stage en entreprise que j'avais fait l'année précédente.

Sur le plan technique, il m'a permis d'approfondir les modèles de séries temporelles, la décomposition saisonnière, les méthodes d'adaptation de domaine et de me familiariser avec les bibliothèques statsmodels et ADAPT.

Abréviations et notations générales

ACF Fonction d'autocorrélation. Mesure la corrélation d'une série avec elle-même décalée d'un certain nombre de pas.

AIC Critère d'information d'Akaike. Critère de sélection de modèle arbitrant entre qualité d'ajustement et nombre de paramètres.

AR Autorégressif. Modèle où la valeur courante dépend linéairement de ses propres valeurs passées.

ARIMA Modèle combinant une partie autorégressive, une différenciation et une moyenne mobile.

HAR Human Activity Recognition. Nom du jeu de données de reconnaissance d'activité utilisé.

MA Moyenne mobile. Modèle où la valeur courante dépend des aléas passés.

MSE, RMSE Erreur quadratique moyenne et sa racine.

MSTL Décomposition saisonnière multiple par lissages locaux successifs.

STL Seasonal-Trend decomposition using Loess.

Table des matières

Préambule **1**

Abréviations et notations générales **2**

1 Introduction **6**

2 Environnement et problématique **6**
2.1 Organisme et équipe d'accueil . . . 6
2.2 Problématique générale . . . 7
2.3 Le changement de distribution . . . 7
2.4 Ce que ces catégories deviennent sur une série temporelle . . . 8
2.5 Enjeux et intérêt du stage . . . 8
2.6 Démarche et étapes . . . 8

3 Positionnement et outils **9**
3.1 État de l'art et positionnement . . . 9
3.2 Notations . . . 9
3.3 Concepts mobilisés : séries temporelles . . . 10
3.3.1 Stationnarité . . . 10
3.3.2 Autocorrélation . . . 11
3.3.3 L'opérateur retard . . . 11
3.4 Le modèle autorégressif . . . 11
3.4.1 Définition . . . 11
3.4.2 Condition de stationnarité . . . 12
3.4.3 Le cas AR(1) . . . 12
3.5 Le modèle à moyenne mobile . . . 13
3.5.1 Définition . . . 13
3.5.2 La difficulté propre au MA . . . 13
3.6 Le modèle ARIMA . . . 13
3.6.1 ARMA . . . 13
3.6.2 Différenciation et ARIMA . . . 13
3.7 Le critère d'information d'Akaike . . . 14
3.8 Prévoir à 24 heures avec un modèle à un pas . . . 15
3.9 Décomposition d'une série . . . 15
3.9.1 Le modèle additif . . . 15
3.9.2 Le principe de STL . . . 15
3.9.3 MSTL . . . 16
3.9.4 Deux propriétés aux conséquences pratiques . . . 16
3.10 Passage à un problème supervisé . . . 16
3.11 Le modèle de référence : la régression sur retards . . . 17
3.11.1 La somme des coefficients . . . 17
3.12 Les méthodes d'adaptation de domaine . . . 17
3.12.1 Les 3 familles . . . 17
3.12.2 Les méthodes retenues . . . 17
3.13 Les métriques . . . 18
3.13.1 Pour la prévision . . . 18
3.13.2 Pour la classification . . . 18
3.14 Les protocoles de validation . . . 18
3.15 Outils logiciels . . . 19

4 Travaux réalisés **20**
4.1 La fabrication les décalages . . . 20
4.1.1 Le générateur . . . 20
4.1.2 Le pipeline de référence . . . 20
4.2 Le choix des jeux réels . . . 20
4.3 La prévision de température . . . 21
4.3.1 Les données . . . 21
4.3.2 Deux versions du problème . . . 21
4.3.3 Retirer la saisonnalité sans tricher . . . 22
4.3.4 Un défaut de données . . . 22
4.3.5 Les modèles comparés . . . 23
4.4 La classification d'activité . . . 24
4.4.1 Les données . . . 24
4.4.2 Chercher où le modèle se trompe . . . 27
4.4.3 Scénario réaliste . . . 27
4.5 Organisation du code . . . 27

5 Résultats **28**
5.1 Sur données synthétiques . . . 28
5.1.1 Vérifier l'extraction avant de s'en servir . . . 28
5.1.2 Tous les décalages ne se valent pas . . . 28
5.1.3 9 méthodes . . . 29
5.1.4 Un modèle qui se rabat sur la moyenne qu'il a apprise . . . 30
5.1.5 Un sixième décalage : la mémoire du bruit . . . 31
5.2 Sur la prévision de température . . . 32
5.2.1 Le décalage n'apparaît qu'après désaisonnalisation . . . 32
5.2.2 Ce que mesure vraiment l'erreur normalisée . . . 33
5.2.3 Un simple recentrage bat deux des 3 méthodes . . . 34
5.2.4 Travailler sur les variations . . . 35
5.2.5 Ce que vaut une prédiction sans modèle . . . 36
5.2.6 Régression contre ARIMA . . . 36
5.2.7 Jusqu'où remonter dans le passé . . . 37
5.2.8 Un piège dont j'ai été victime . . . 38
5.3 Sur la classification d'activité . . . 38
5.3.1 Le score global masque les difficultés . . . 38
5.3.2 La frontière mobile-immobile . . . 39
5.3.3 Deux sujets difficiles, deux causes différentes . . . 39
5.3.4 Un sujet difficile à prédire est un bon sujet source . . . 40
5.3.5 Combien de données cible faut-il . . . 41
5.3.6 L'expérience du nouveau sujet . . . 43
5.3.7 Les trois familles d'ADAPT, vues de loin puis de près . . . 43
5.3.8 Pourquoi CORAL échoue . . . 44

6 Écrire les décalages pour des variables non indépendantes **45**
6.1 Ce que les articles écrivent . . . 45
6.2 Deux dépendances à distinguer . . . 45
6.3 En classification . . . 46
6.4 En prévision . . . 46
6.5 Les 6 décalages écrits en équation . . . 46
6.6 Où se placent les 2 décalages réels . . . 47

7 Conclusion **48**
7.1 Ce que j'ai fait . . . 48
7.2 Ce qui reste incomplet . . . 48
7.3 Perspectives . . . 48
7.4 Ce que je laisse . . . 48

Bibliographie **48**

Glossaire **49**

A Annexe : organisation du code **50**

B Annexe : les jeux de données **51**

C Annexe : supports de cours utilisés **51**

D Annexe : pages de référence en ligne **51**

E Annexe : les décalages synthétiques **52**

F Annexe DD&RS : développement durable et responsabilité sociétale **52**
F.1 Périmètre et sources . . . 52
F.2 Pressions exercées sur l'environnement . . . 52
F.3 Cadre réglementaire . . . 53
F.4 Politique de l'établissement . . . 54
F.5 Lien entre la mission et la sobriété . . . 54
F.6 Responsabilité sociétale . . . 54
F.7 Analyse critique . . . 54
F.8 Sources . . . 55

G Annexe : déclaration d'utilisation de l'intelligence artificielle **55**

H Annexe : quatre démonstrations **55**
H.1 L'espérance conditionnelle minimise l'erreur quadratique . . . 55
H.2 Variance de l'erreur de prévision d'un AR(1) . . . 56
H.3 Le seuil d'acceptation d'un paramètre par l'AIC . . . 57
H.4 Le biais de transfert d'une régression sur retards . . . 57

Liste des figures **59**

1 Introduction

J'ai effectué mon stage de deuxième année à l'ENSIIE, sous l'encadrement de Mounir Atiq, enseignant-chercheur au CEA. Il s'inscrit dans les travaux menés autour de la bibliothèque Python ADAPT, dont mon encadrant est l'un des auteurs, aux côtés de Mathilde Mougeot, mon enseignante référente.

Le sujet porte sur l'adaptation de domaine appliquée aux séries temporelles. Un modèle est entraîné sur des données dites source, puis employé sur des données dites cible qui ne suivent pas exactement la même distribution. Sur des séries temporelles, ce cas est la règle plutôt que l'exception : les conditions environnementales évoluent, les capteurs changent, la population observée n'est pas la même. Les performances se dégradent alors, parfois fortement.

L'objectif du stage était double : construire un banc d'essai comparant les méthodes d'adaptation de la bibliothèque ADAPT sur des séries temporelles, et caractériser les types de changement de distribution que l'on rencontre en pratique, afin de déterminer si certaines méthodes conviennent mieux à certains types de décalage.

Le travail s'est appuyé sur des séries synthétiques, dont je contrôlais le décalage, puis sur 2 jeux de données réels : des relevés horaires de température sur plusieurs villes, pour une tâche de prévision, et le jeu UCI HAR de reconnaissance d'activité humaine, pour une tâche de classification. Dans le premier cas le domaine est la ville, dans le second le sujet portant le capteur.

J'ai travaillé en Python, avec scikit-learn pour les modèles de référence, statsmodels pour la décomposition et la modélisation de séries, et ADAPT pour les méthodes d'adaptation. L'ensemble est organisé en notebooks Jupyter appuyés sur 2 modules partagés.

Plan du rapport. Je commence par présenter l'environnement du stage et la problématique, en précisant ce qu'est un changement de distribution et ce que cette notion devient sur une série temporelle. Je décris ensuite les notations que je conserve dans le rapport, ainsi que les concepts, modèles et méthodes mobilisés. Je présente alors les travaux réalisés, phase par phase. J'expose enfin les résultats obtenus et leur discussion, avant de conclure sur ce qui a été atteint et sur les perspectives.

2 Environnement et problématique

2.1 Organisme et équipe d'accueil

Ce stage de recherche s'est déroulé à l'École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise, à Évry-Courcouronnes, du 1er juillet au 25 août 2026. Les travaux ont eu lieu dans les locaux du C19, l'incubateur adossé à l'école, qui accueille des projets numériques et sert de lieu de travail aux stagiaires.



FIGURE 1 – L'espace de travail partagé du C19, où s'est déroulé le stage.

Le stage a été encadré par Mounir Atiq, enseignant-chercheur, et suivi par Mathilde Mougeot comme enseignante référente. Tous deux comptent parmi les auteurs de la bibliothèque ADAPT [1], qui est précisément l'objet de ce stage.

Nous étions un petit groupe de 2 élèves de deuxième année encadrés par la même personne, sur des sujets distincts. Cette configuration m'a laissé une large autonomie dans la conduite des expériences, tout en permettant des échanges techniques quotidiens sur les outils communs, l'organisation du code et les protocoles d'évaluation. L'encadrement a pris la forme de points d'étape réguliers, chacun ouvrant en général une nouvelle question à mesurer.

2.2 Problématique générale

L'apprentissage supervisé repose sur une hypothèse qu'on énonce rarement. On suppose que les données d'entraînement et les données d'usage viennent de la même loi. Quand ce n'est pas le cas, le modèle continue de prédire, sans rien signaler, mais ses prédictions se dégradent.

La problématique de ce stage peut donc se formuler ainsi : *comment évaluer, sur des séries temporelles, les méthodes qui prétendent corriger un écart de distribution entre le domaine d'entraînement et le domaine d'usage, et comment savoir laquelle employer selon la nature de cet écart ?*

2.3 Le changement de distribution

On note $\mathcal{S}$ le domaine source, celui sur lequel on dispose de données annotées, et $\mathcal{T}$ le domaine cible, celui sur lequel on veut prédire. L'hypothèse usuelle s'écrit

$$P_{\mathcal{S}}(X, Y) = P_{\mathcal{T}}(X, Y).$$

Lorsqu'elle est fausse, on parle de changement de distribution. En décomposant la loi jointe de deux manières différentes, on distingue trois situations classiques, illustrées figure 2.

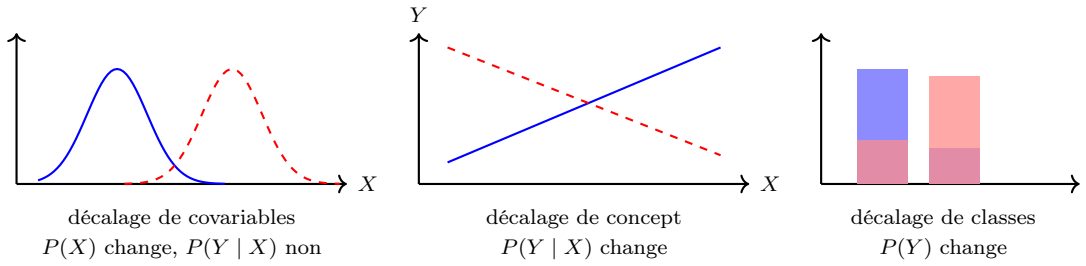


FIGURE 2 – Les trois formes de changement de distribution. En trait plein le domaine source, en pointillés le domaine cible.

Le décalage de covariables. La loi des entrées change, celle de la sortie sachant l'entrée reste la même :

$$P_{\mathcal{S}}(X) \neq P_{\mathcal{T}}(X), \qquad P_{\mathcal{S}}(Y \mid X) = P_{\mathcal{T}}(Y \mid X).$$

Le décalage de concept. La loi des entrées ne change pas, mais la relation elle-même change :

$$P_{\mathcal{S}}(X) = P_{\mathcal{T}}(X), \qquad P_{\mathcal{S}}(Y \mid X) \neq P_{\mathcal{T}}(Y \mid X).$$

Le décalage de classes. Les proportions des classes changent.

2.4 Ce que ces catégories deviennent sur une série temporelle

Sur une série temporelle, ces catégories se traduisent en phénomènes que l'on peut nommer, et surtout fabriquer. C'est ce qui a guidé la première phase du stage. J'en ai retenu cinq, présentés figure 3.

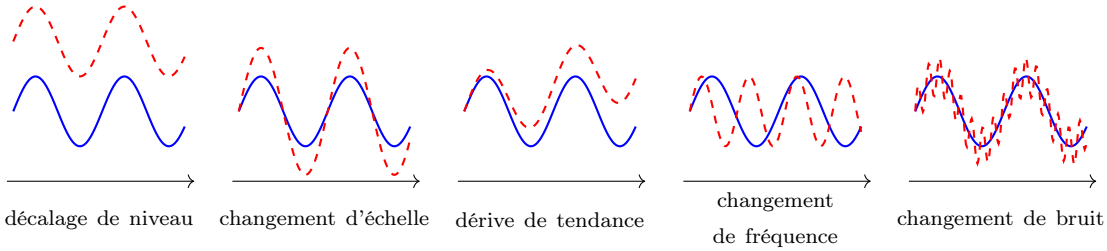


FIGURE 3 – Les 5 décalages fabriqués sur données synthétiques. Trait plein bleu : domaine source. Pointillés rouges : domaine cible.

nom	ce qui change
décalage de niveau	la moyenne de la série
changement d'échelle	l'amplitude
dérive de tendance	la pente
changement de fréquence	la période du cycle
changement de bruit	la variance de l'aléa

Ces 5 décalages sont nommés d'après ce qu'ils modifient dans la série, et non d'après les 3 formes précédentes. Les rattacher à ces formes demande une écriture plus précise, que je donne en section 6.

Le décalage de classes, lui, n'apparaît pas ici. Il n'a de sens qu'en classification, alors que ces 5 décalages ont été construits sur une tâche de prévision.

2.5 Enjeux et intérêt du stage

La bibliothèque ADAPT [1] met à disposition une trentaine de méthodes d'adaptation, compatible avec scikit-learn. Son évaluation sur des séries temporelles restait en revanche peu documentée.

2.6 Démarche et étapes

Le sujet prévoyait 5 étapes :

1. prise en main de la bibliothèque ADAPT et revue des principales approches d'adaptation ;
2. sélection de jeux de données réels soumis à un changement de domaine, et génération de données synthétiques ;
3. mise en œuvre d'un pipeline de référence sans adaptation de domaine ;
4. expérimentation de plusieurs méthodes d'adaptation de la bibliothèque ;
5. comparaison quantitative des performances selon le type de changement et le niveau de décalage.

Je les ai toutes abordées, mais pas dans cet ordre et pas une fois chacune. Le travail a été plus itératif et c'est de ses détours que viennent la plupart des résultats de ce rapport.

La dernière étape n'a de sens complet que sur les séries synthétiques, seul cadre où l'on choisit à la fois le type de décalage et son intensité.

3 Positionnement et outils

3.1 État de l'art et positionnement

Une revue de l'adaptation non supervisée sur données de capteurs [7] classe les méthodes en 4 familles, dont 2 recoupent celles d'ADAPT : l'adaptation de l'espace des variables correspond à CORAL, celle par contrainte sur les paramètres à RegularTransfer. Elle porte cependant sur des méthodes profondes, là où je travaille avec des modèles linéaires.

Un banc d'essai compare 11 méthodes sur 5 jeux de données et 50 scénarios de transfert [8]. C'est, en plus grand, ce que ce stage cherchait à construire.

RevIN [15] propose de retirer à chaque fenêtre sa propre moyenne et son propre écart-type avant de la donner au modèle, puis de les restituer sur la sortie. Le recentrage par ville que j'utilise est la même idée, appliquée au domaine plutôt qu'à la fenêtre.

Quatre autres articles écrivent le décalage pour des séries plutôt que pour des observations indépendantes. Ils servent de base à la section 6.

RAINCOAT [11] traite ensemble le décalage des variables et celui des classes, en comparant les séries à la fois dans le temps et en fréquence. L'article [12] traite le décalage de covariables en classification par transport optimal. Tous les 2 prennent la fenêtre entière comme observation, ce qui leur permet de garder les définitions habituelles.

L'article [9] est le seul à s'attaquer à la prévision, où ce raccourci n'est plus possible. Il montre que l'hypothèse de décalage de covariables ne se transpose pas et propose de fixer la structure des dépendances entre variables tout en laissant leur intensité varier. L'article [13] part du même constat, à savoir que les séries réelles ne sont pas des observations indépendantes, et modélise la dérive au fil du temps plutôt qu'entre 2 domaines. L'article [14] traite enfin la dérive de concept en prévision, du côté de l'apprentissage en ligne plutôt que de l'adaptation de domaine.

Ce travail se distingue de ces articles sur plusieurs plans. Ils portent presque tous sur la classification, alors que j'ai traité aussi la prévision. Ils emploient des réseaux profonds, alors que mes régressions linéaires rendent les mécanismes lisibles, comme le montre la somme des coefficients. Et deux de mes observations portent sur le protocole d'évaluation plutôt que sur les méthodes : la lecture trompeuse du coefficient de détermination entre domaines de variances différentes, et le fait que la saisonnalité commune masque le décalage tant qu'on ne l'a pas retirée.

3.2 Notations

Je fixe ici les notations utilisées dans tout le rapport.

symbole	signification
$x = (x_t)_{1 \le t \le n}$	la série observée, de longueur n
n_{app}	indice de la coupure entre apprentissage et test
$\tau_t,\ s_t,\ r_t$	tendance, saisonnalité et résidu, tels que $x_t = \tau_t + s_t + r_t$
$z_t = x_t - s_t$	la série désaisonnalisée, égale à $\tau_t + r_t$
$\eta_t = z_t - \mu$	la série centrée sur sa moyenne
$\hat{\sigma}_{\text{app}},\ \hat{\sigma}_{\text{test}}$	écart-type du résidu r, sur l'apprentissage et sur le test
w	largeur de la fenêtre d'entrée
h	horizon de prévision
$X_i = (\eta_i, \dots, \eta_{i+w-1})$	la i-ième fenêtre d'entrée, dans $\mathbb{R}^w$
$y_i = \eta_{i+w+h-1}$	la valeur à prédire pour cette fenêtre
$\mathcal{S},\ \mathcal{T}$	domaines source et cible
f	la mémoire de la série, dans l'écriture de la section 6
$g(t)$	partie déterministe de la série, tendance et saisonnalité

symbole	signification
$\delta(t) = g_{\mathcal{T}}(t) - g_{\mathcal{S}}(t)$	écart entre les parties déterministes des 2 domaines
$x_{(p)}$	les p dernières coordonnées d'une fenêtre
$a_1, \dots, a_w$ et b	coefficients et ordonnée à l'origine de la régression
B	opérateur retard, défini par $B\,u_t = u_{t-1}$
$p,\ q$	ordres autorégressif et de moyenne mobile
$\varphi_1, \dots, \varphi_p$	coefficients autorégressifs
$\theta_1, \dots, \theta_q$	coefficients de moyenne mobile
$\varphi(B),\ \theta(B)$	polynômes autorégressif et de moyenne mobile
μ	moyenne du processus
ε_t	innovation, ou aléa, de variance σ^2
$\rho(k)$	autocorrélation au décalage k
λ	force du rappel vers le modèle source

Moyenne et ordonnée à l'origine. Le symbole μ désigne la moyenne du processus, et non l'ordonnée à l'origine de la récursion autorégressive.

La distinction vient de ce qu'il existe deux façons d'écrire le même modèle. La première porte sur la série centrée $\eta_t = z_t - \mu$ et ne comporte aucune constante, puisque la moyenne a été retirée en amont. La seconde porte directement sur z, et une constante c y apparaît :

$$z_t = c + \varphi_1 z_{t-1} + \cdots + \varphi_p z_{t-p} + \varepsilon_t.$$

On obtient la relation entre les deux en prenant l'espérance de cette seconde écriture. La série est stationnaire, donc $\mathbb{E}[z_t] = \mathbb{E}[z_{t-j}] = \mu$, et l'aléa est centré. Il reste

$$\mu = c + \mu \sum_{j=1}^{p} \varphi_j, \qquad \text{d'où} \qquad c = \mu \left(1 - \sum_{j=1}^{p} \varphi_j \right).$$

Les deux quantités peuvent donc être très éloignées l'une de l'autre, d'autant plus que $\sum_j \varphi_j$ est proche de 1. La bibliothèque statsmodels retient la première écriture et nomme son paramètre `const` : celui-ci vaut μ, et non c. Confondre les deux donne un modèle faux sans qu'aucune erreur ne soit signalée, ce qui a des conséquences pratiques développées plus loin.

3.3 Concepts mobilisés : séries temporelles

3.3.1 Stationnarité

Une série (z_t) est stationnaire au second ordre lorsque trois conditions sont réunies : son espérance ne dépend pas du temps, sa variance est finie et constante, et la covariance entre deux instants ne dépend que de leur écart. On écrit :

$$\mathbb{E}[z_t] = \mu, \qquad \mathrm{Var}(z_t) = \gamma(0), \qquad \mathrm{Cov}(z_t, z_{t+k}) = \gamma(k).$$

C'est l'hypothèse de base des modèles autorégressifs. Elle garantit que ce que le modèle apprend sur une portion de la série vaut aussi sur les autres. Une série qui dérive n'est pas stationnaire, mais ses variations peuvent l'être.

3.3.2 Autocorrélation

L'autocorrélation au décalage k est

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)} = \frac{\mathrm{Cov}(z_t, z_{t+k})}{\mathrm{Var}(z_t)}.$$

Elle est sans dimension et comprise entre -1 et 1. Sur une température horaire, $\rho(1)$ vaut environ 0,99, et $\rho(24)$ reste élevée à cause du cycle journalier.

L'autocorrélation sert ici à détecter la période d'un cycle en repérant les décalages où elle culmine. (ACF)

C'est ainsi que les 2 périodes des séries de température ont été obtenues. L'autocorrélation présente des pics nets à 24, 48 et 72 heures, ce qui donne le cycle journalier. Pour le cycle annuel, l'autocorrélation ne suffit pas : il faudrait des décalages de plusieurs milliers de pas. J'ai utilisé un périodogramme, qui décompose la variance de la série par fréquence et fait ressortir les périodes dominantes. Il donne 24 heures et une période longue voisine de 8 892 heures.

Un périodogramme ne peut renvoyer que des périodes de la forme n/k avec k entier, et sur nos 44 460 points, $44\,460/5 = 8\,892$ est le pas disponible le plus proche d'une année. La valeur 8 766 finalement retenue pour la décomposition ne vient donc pas du signal mais du calendrier : elle correspond à 365,25 jours.

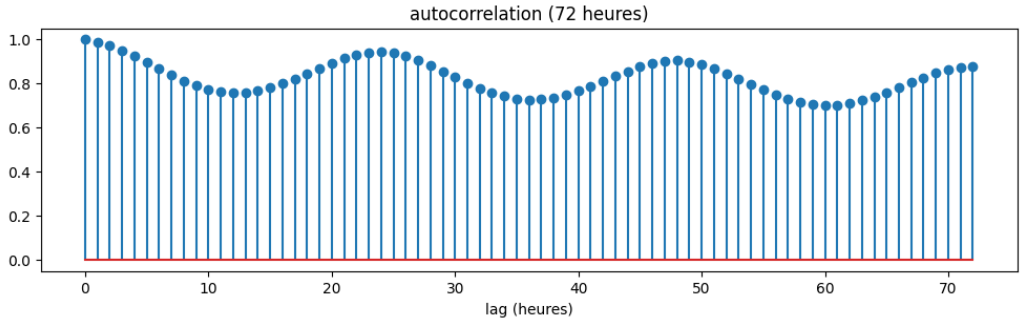


FIGURE 4 – Autocorrélation de la température horaire après retrait de la tendance. Les pics à 24, 48 et 72 heures révèlent le cycle journalier.

3.3.3 L'opérateur retard

Pour écrire les modèles de façon compacte, on introduit l'opérateur retard B, défini par

$$B\,u_t = u_{t-1}, \qquad \text{donc} \qquad B^k u_t = u_{t-k}.$$

3.4 Le modèle autorégressif

3.4.1 Définition

Un processus autorégressif d'ordre p, noté AR(p), s'écrit sur la série centrée $\eta_t = z_t - \mu$:

$$\boxed{\eta_t = \varphi_1 \eta_{t-1} + \varphi_2 \eta_{t-2} + \cdots + \varphi_p \eta_{t-p} + \varepsilon_t}$$

où (ε_t) est un bruit blanc de variance σ^2, c'est-à-dire une suite de variables centrées, de même variance et non corrélées entre elles.

En introduisant le polynôme autorégressif

$$\varphi(B) = 1 - \varphi_1 B - \varphi_2 B^2 - \cdots - \varphi_p B^p,$$

le modèle s'écrit simplement $\varphi(B)\,\eta_t = \varepsilon_t$.

3.4.2 Condition de stationnarité

Le processus est stationnaire si et seulement si toutes les racines du polynôme $\varphi(B) = 0$ sont de module strictement supérieur à un. Pour un AR(1), cette condition se réduit à $|\varphi_1| < 1$.

3.4.3 Le cas AR(1)

On pose $\eta_t = \varphi\,\eta_{t-1} + \varepsilon_t$ avec $|\varphi| < 1$.

Variance. En prenant la variance des deux membres et en utilisant l'indépendance entre η_{t-1} et ε_t :

$$\gamma(0) = \varphi^2\gamma(0) + \sigma^2 \qquad \Longrightarrow \qquad \gamma(0) = \frac{\sigma^2}{1-\varphi^2}.$$

Autocorrélation. En multipliant par η_{t-k} et en prenant l'espérance, $\gamma(k) = \varphi\,\gamma(k-1)$, donc

$$\rho(k) = \varphi^{\,k}.$$

Prévision à h pas. On cherche la fonction du passé qui minimise l'erreur quadratique moyenne. Cette fonction est l'espérance conditionnelle $\mathbb{E}[\eta_{t+h} \mid \mathcal{F}_t]$, où $\mathcal{F}_t$ désigne tout ce qui est connu à l'instant t. Comme l'aléa futur est centré et indépendant du passé, la calculer revient simplement à remplacer chaque ε futur par zéro :

$$\hat\eta_{t+1} = \varphi\,\eta_t, \qquad \hat\eta_{t+2} = \varphi\,\hat\eta_{t+1} = \varphi^2\eta_t, \qquad \dots \qquad \boxed{\hat\eta_{t+h} = \varphi^{\,h}\,\eta_t}$$

Pour prédire h pas en avant, il suffit de multiplier la valeur courante par une seule constante, $\varphi^{\,h}$, qui ne dépend que de l'horizon. Aucune autre information n'intervient.

Or cette constante s'écrase très vite lorsque l'horizon s'allonge :

φ	0,50	0,80	0,90	0,95	0,99
φ^{24}	0,0000	0,0047	0,0798	0,2920	0,7857

Avec $\varphi = 0{,}9$, la prévision à 24 pas ne conserve que 8 % de l'écart actuel à la moyenne : elle prédit donc presque μ. Le modèle abandonne l'information du présent et se rabat sur le niveau moyen de la série.

Cette propriété signifie qu'un prédicteur à horizon lointain revient vers la moyenne du domaine sur lequel il a été appris. Appliqué à un domaine dont la moyenne est ailleurs, il ramène donc les prédictions vers une valeur qui n'est pas la bonne.

Pour un ordre p supérieur à un, la prévision n'est plus un multiple de la seule valeur courante mais une combinaison linéaire des p dernières valeurs.

Variance de l'erreur de prévision. On déroule la récursion pour faire apparaître ce que la prévision ignore :

$$\begin{aligned}
\eta_{t+1} &= \varphi\,\eta_t + \varepsilon_{t+1},\\
\eta_{t+2} &= \varphi^2\eta_t + \varphi\,\varepsilon_{t+1} + \varepsilon_{t+2},\\
&\;\;\vdots\\
\eta_{t+h} &= \underbrace{\varphi^{\,h}\eta_t}_{\text{connu en } t} \;+\; \underbrace{\sum_{j=0}^{h-1}\varphi^{\,j}\,\varepsilon_{t+h-j}}_{\text{chocs à venir}}.
\end{aligned}$$

Le premier terme est la prévision. Le second accumule tous les chocs qui surviendront entre t et $t+h$, et c'est exactement ce qu'on ne peut pas connaître. L'erreur de prévision est donc ce second terme, d'où

$$\mathrm{Var}\big(\eta_{t+h} - \hat{\eta}_{t+h}\big) = \sigma^2 \, \frac{1-\varphi^{2h}}{1-\varphi^2}.$$

Elle croît avec l'horizon et tend vers la variance $\gamma(0)$ du processus : à horizon lointain, prédire ne vaut pas mieux que donner la moyenne. Cette formule me sert dans un des notebooks pour calculer le plancher de bruit du décalage de mémoire, c'est-à-dire ce qu'un modèle peut faire au mieux quand le bruit est autocorrélé (cf section 5.1.5).

3.5 Le modèle à moyenne mobile

3.5.1 Définition

Un processus à moyenne mobile d'ordre q, noté MA(q), s'écrit

$$\boxed{\eta_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}}$$

soit $\eta_t = \theta(B)\,\varepsilon_t$ avec $\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$.

La valeur courante ne dépend pas des valeurs passées mais des *chocs* passés : une perturbation subie à l'instant $t-1$ continue de se faire sentir à l'instant t, avec le poids θ_1.

3.5.2 La difficulté propre au MA

Les ε_t ne sont pas observés. On voit z_t, pas le choc qui l'a produit. Il faut donc les reconstruire récursivement. Le procédé est le suivant : à chaque instant, on compare ce que le modèle avait prédit à ce que l'on observe, et l'écart fournit l'estimation du choc.

$$\hat{\varepsilon}_t = z_t - \hat{z}_{t|t-1}.$$

Conséquence sur la prévision. Pour un MA(1), la prévision à un pas vaut $\theta_1 \hat{\varepsilon}_t$, mais celle à deux pas est nulle, puisqu'elle ferait intervenir $\mathbb{E}[\varepsilon_{t+1}] = 0$. Un MA($q$) n'a donc aucune mémoire au-delà de q pas. À l'horizon de 24 pas utilisé ici, les termes de moyenne mobile n'apportent rien.

3.6 Le modèle ARIMA

3.6.1 ARMA

En combinant les deux familles, on obtient un processus ARMA(p, q) :

$$\varphi(B)\,\eta_t = \theta(B)\,\varepsilon_t.$$

3.6.2 Différenciation et ARIMA

Un modèle ARMA suppose la stationnarité. Quand la série dérive, on modélise ses variations. On note $\Delta z_t = z_t - z_{t-1} = (1-B) z_t$, et plus généralement $(1-B)^d$ pour une différenciation d'ordre d.

Le modèle ARIMA(p, d, q) s'écrit alors

$$\boxed{\varphi(B)\,(1-B)^d\, z_t = c + \theta(B)\,\varepsilon_t}$$

Une conséquence de notation. Avec $d = 0$, la bibliothèque statsmodels inclut par défaut un terme constant représentant la moyenne μ. Avec $d \geq 1$, la différenciation supprime le niveau et ce terme disparaît. Deux modèles d'ordres $(2, 0, 0)$ et $(2, 1, 1)$ ont donc le même nombre de paramètres, quatre, mais pas la même signification :

$$(2,0,0) : (\mu, \varphi_1, \varphi_2, \sigma^2) \qquad \text{contre} \qquad (2,1,1) : (\varphi_1, \varphi_2, \theta_1, \sigma^2).$$

Transférer un vecteur de paramètres de l'un vers l'autre par position décale tous les rôles d'un cran. La bibliothèque l'accepte sans erreur et la prévision diverge. J'ai rencontré ce problème et j'y reviens dans les résultats.

3.7 Le critère d'information d'Akaike

Les bibliothèques affichent pour chaque modèle ajusté une valeur appelée AIC. Comme je m'en sers pour comparer des ordres, je précise ce qu'elle mesure.

Pour comparer deux modèles, on aimerait savoir lequel prédira le mieux des données nouvelles. On ne dispose que des données d'apprentissage, et la qualité de l'ajustement sur ces données est un mauvais juge : un modèle avec plus de paramètres colle toujours mieux à ce qu'il a déjà vu, même quand les paramètres supplémentaires ne servent à rien.

Akaike a montré que, sous certaines conditions, cet excès vaut approximativement le nombre de paramètres estimés. Le critère s'écrit

$$\mathrm{AIC} = 2k - 2 \ln \hat{L},$$

où k est le nombre de paramètres estimés et $\hat{L}$ la vraisemblance maximisée, c'est-à-dire la probabilité que le modèle attribue aux données observées une fois ses paramètres réglés au mieux. Le terme $-2 \ln \hat{L}$ récompense l'ajustement, le terme $2k$ paie le prix des paramètres. On préfère l'AIC le plus faible.

Pour un modèle à aléas gaussiens de variance résiduelle $\hat{\sigma}^2$ estimée sur n points, la vraisemblance prend une forme explicite et le critère devient, à une constante près qui ne dépend pas du modèle,

$$\mathrm{AIC} = n \ln \hat{\sigma}^2 + 2k + \text{constante}.$$

C'est la forme utile : l'AIC échange le logarithme de la variance résiduelle contre le nombre de paramètres. Seules les différences d'AIC entre modèles ont un sens, la valeur absolue n'en a aucune.

Trois limites comptent pour la suite de ce rapport.

L'AIC porte sur l'ajustement à un pas. La vraisemblance d'un ARIMA est construite sur les erreurs de prévision à un pas. Mon erreur de référence porte sur 24 pas. Les deux quantités ne sont pas la même, et rien ne garantit qu'elles classent les modèles dans le même ordre.

L'AIC ne regarde jamais les données de test. C'est un critère d'apprentissage : il estime la performance future à partir de la seule période d'apprentissage.

La pénalité est faible quand n est grand. Passer de k à $k + 1$ paramètres augmente la pénalité de exactement 2, quels que soient les données et le paramètre ajouté. Dans le même temps le terme d'ajustement $n \ln \hat{\sigma}^2$ diminue, puisque le modèle d'ordre $p + 1$ contient celui d'ordre p comme cas particulier, obtenu en annulant le coefficient supplémentaire, et qu'il ne peut donc pas s'ajuster moins bien. L'AIC du modèle le plus riche est donc le plus faible dès que cette diminution dépasse 2. Comme le terme d'ajustement est proportionnel à n, il suffit pour cela d'une baisse relative de $\hat{\sigma}^2$ de l'ordre de $2/n$. Avec les 35 568 points d'apprentissage de la ville cible, cela représente 0,006 %. Le seuil est donc dérisoire, et c'est ce qui explique que l'AIC continue de décroître jusqu'au bout de la grille.

3.8 Prévoir à 24 heures avec un modèle à un pas

Un modèle ARIMA décrit un seul pas. Pour prévoir 24 heures en avant, on applique sa règle, puis on la réapplique à son propre résultat, 24 fois de suite.

Le modèle a besoin de son point de départ, c'est-à-dire de l'état du processus à l'instant depuis lequel on prédit. La bibliothèque le reconstitue en rejouant le modèle sur les observations passées.

La comparaison avec la régression impose une contrainte. La régression produit une prédiction par fenêtre de test, soit 8 883 en tout, et l'erreur porte sur l'ensemble. L'ARIMA doit donc produire 8 883 prévisions, une depuis chaque instant du test. On appelle origine l'instant depuis lequel une prévision est faite : c'est le dernier point connu.
Redonner tout l'historique à chaque origine coûte plusieurs dizaines de minutes par modèle ce qui rend le balayage des ordres trop long. Or l'état d'un ARIMA ne dépend que de ses dernières valeurs, et un court préfixe suffit à le reconstituer. Je ne fournis donc que les 200 points précédant chaque origine, ce qui ramène le coût à quelques secondes. Ce raccourci n'est pas une approximation : j'ai vérifié, pour les six ordres testés, que la prévision obtenue est rigoureusement identique à celle obtenue avec l'historique complet.

3.9 Décomposition d'une série

3.9.1 Le modèle additif

Une série réelle mélange plusieurs échelles de temps. La décomposition additive sépare

$$x_t = \tau_t + s_t + r_t,$$

où τ varie lentement, s est périodique et r est le reste.

3.9.2 Le principe de STL

La décomposition STL [4] repose sur le lissage local, appelé Loess. Pour lisser au point t, on prend les voisins les plus proches, on les pondère selon leur distance, on ajuste un polynôme sur eux et on l'évalue en t. Chaque valeur lissée est donc une petite régression locale. C'est ce qui permet à la forme de changer lentement le long de la série.

La boucle interne enchaîne 6 étapes :

1. détendancer, en retirant l'estimation courante de τ ;
2. lisser les sous-séries cycliques : pour une période de 24 heures, on regroupe toutes les valeurs de minuit ensemble, toutes celles de une heure ensemble, et l'on lisse chaque groupe séparément ;
3. filtrer en passe-bas ces sous-séries lissées, par 3 moyennes mobiles suivies d'un lissage. Cette étape extrait ce que les sous-séries contiennent de basse fréquence, c'est-à-dire une dérive lente qui appartient à la tendance et non au cycle. Sans elle, la composante saisonnière absorberait une partie de la tendance ;
4. obtenir la saisonnalité en soustrayant le passe-bas des sous-séries lissées, ce qui lui garantit une moyenne nulle sur chaque cycle ;
5. désaisonnaliser, en retirant s de la série ;
6. estimer la tendance en lissant la série désaisonnalisée.

Une boucle externe repondère ensuite les points selon leur résidu, ce qui rend la décomposition robuste aux valeurs aberrantes.

3.9.3 MSTL

Lorsque la série comporte plusieurs périodes, comme un cycle journalier et un cycle annuel, MSTL applique STL successivement à chaque période, de la plus courte à la plus longue, et itère l'ensemble. À chaque tour, la composante en cours est remise dans la série avant d'être réestimée, de sorte que chaque décomposition voit la série débarrassée des autres saisonnalités.

3.9.4 Deux propriétés aux conséquences pratiques

La saisonnalité n'est pas un motif fixe. L'étape de lissage des sous-séries cycliques lui permet d'évoluer lentement. Il n'existe donc pas une saisonnalité unique à recopier, et prolonger la décomposition au-delà de la période d'apprentissage oblige à choisir quelle version prolonger.

Le lissage est bilatéral. Chaque valeur lissée utilise des voisins des deux côtés. Ajuster une décomposition sur l'ensemble d'une série laisse donc les données de test influencer l'estimation partout, y compris sur la partie apprentissage. C'est une fuite d'information qu'il faut éviter.

3.10 Passage à un problème supervisé

Une série n'est pas directement un jeu supervisé. Je la découpe en fenêtres glissantes : la fenêtre X_i rassemble w valeurs consécutives, et la cible y_i est la valeur observée h pas après la fin de cette fenêtre.

$$X_i = (\eta_i,\ \eta_{i+1},\ \ldots,\ \eta_{i+w-1}) \in \mathbb{R}^w, \qquad y_i = \eta_{i+w+h-1}.$$

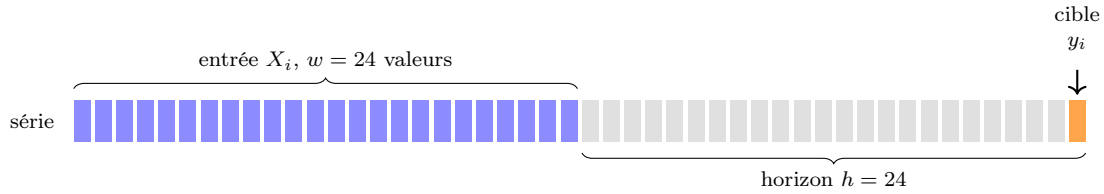


FIGURE 5 – Le découpage en fenêtres glissantes. Le modèle lit les 24 valeurs en bleu et prédit la valeur en orange, située 24 pas après la fin de la fenêtre. Les valeurs grises sont le futur au moment de la prédiction : elles ne sont pas lues.

À $h = 1$, la prévision de température est presque triviale : l'autocorrélation au décalage un vaut environ 0,99, et recopier la dernière valeur suffit à obtenir un très bon score. J'ai retenu $h = 24$, ce qui revient à prédire le lendemain à la même heure.

Le découpage apprentissage-test est chronologique et jamais aléatoire : les 80 % premiers points servent à l'apprentissage, les 20 % derniers au test.

Sur la série désaisonnalisée, ce découpage se fait en réalité deux fois, sur deux objets différents.

La première coupure porte sur les points de la série. Elle sert à la décomposition : celle-ci est ajustée sur les 80 % premiers points, et au-delà la saisonnalité doit être fabriquée. Sur nos données, elle tombe au point 35 568 sur 44 460.

La seconde coupure porte sur les fenêtres. Elle sert à l'évaluation : le modèle apprend sur les 80 % premières fenêtres et il est noté sur les dernières. Or le fenêtrage produit moins de fenêtres qu'il n'y a de points, puisqu'il en faut $w + h - 1 = 47$ pour en construire une seule. On passe donc de 44 460 points à 44 413 fenêtres, et 80 % de ce nombre plus petit tombe un peu plus tôt, à la fenêtre 35 530.

Les deux coupures diffèrent ainsi de 38 indices. L'écart est sans conséquence sur les résultats, mais il explique pourquoi les deux nombres ne coïncident pas.

3.11 Le modèle de référence : la régression sur retards

Le modèle de référence est une régression linéaire sans régularisation :

$$\hat{y} = f(X_i) = \sum_{j=1}^{w} a_j \, \eta_{i+j-1} + b.$$

C'est un prédicteur direct : il est entraîné à minimiser l'erreur exactement à l'horizon h qui nous intéresse, contrairement à un ARIMA qui modélise un pas et l'itère.

3.11.1 La somme des coefficients

Si toutes les valeurs de la fenêtre sont égales à un même niveau L, on peut mettre L en facteur :

$$\hat{y} = \Big(\sum_{j=1}^{w} a_j \Big) \, L + b.$$

Cette somme mesure donc la fraction du niveau d'entrée qui ressort. Si elle vaut un, un décalage constant de l'entrée se retrouve intégralement dans la sortie. Si elle vaut moins que un, il tire la prédiction vers la moyenne de son jeu d'apprentissage.

3.12 Les méthodes d'adaptation de domaine

3.12.1 Les 3 familles

La bibliothèque ADAPT organise les méthodes selon l'endroit où la correction est appliquée.

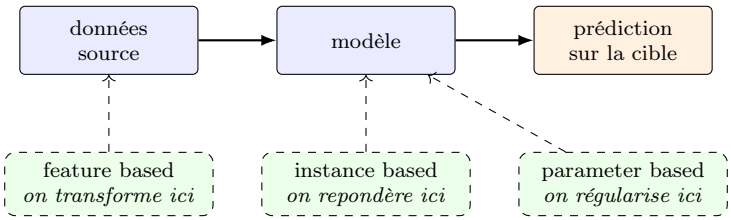


FIGURE 6 – Les 3 familles de méthodes d'adaptation, situées par l'endroit où elles interviennent dans la chaîne.

3.12.2 Les méthodes retenues

Neuf méthodes ont été comparées sur données synthétiques : CORAL, SA, TCA, fMMD, KMM, KLIEP, FA, TrAdaBoostR2 et RegularTransferLR. Sur données réelles, j'en ai retenu trois, une par famille.

CORAL , de la famille feature based, cherche à donner aux entrées source la même structure de corrélation qu'aux entrées cible. Elle applique pour cela une transformation linéaire qui remplace la covariance de la source par celle de la cible :

$$X'_{\mathcal{S}} = X_{\mathcal{S}} \, \Sigma_{\mathcal{S}}^{-1/2} \, \Sigma_{\mathcal{T}}^{1/2}.$$

Le premier facteur ramène la source à une covariance identité, le second lui impose celle de la cible. Elle n'utilise pas les étiquettes cible.

KMM , de la famille instance based, attribue à chaque exemple source un poids ω_i, de sorte que la distribution pondérée de la source ressemble à celle de la cible. Les poids sont obtenus en minimisant une distance entre les moyennes des deux échantillons. Elle n'utilise pas non plus les étiquettes cible.

RegularTransfer , de la famille parameter based, part du modèle source de paramètres $\beta_{\mathcal{S}}$ et résout

$$\hat{\beta} = \arg\min_{\beta} \ \mathcal{L}_{\mathcal{T}}(\beta) + \lambda \, \|\beta - \beta_{\mathcal{S}}\|^2.$$

Un λ grand maintient le modèle près de la source, un λ petit le laisse suivre les données cible. Cette méthode a besoin des étiquettes cible, ce qui la place dans un cadre différent des deux précédentes.

3.13 Les métriques

3.13.1 Pour la prévision

$$R^2 = 1 - \frac{\mathrm{MSE}}{\mathrm{Var}(y)}$$

Son dénominateur dépend du domaine évalué, ce qui le rend délicat à comparer d'un domaine à l'autre. Pour un modèle parfait qui n'atteindrait que le bruit, il vaudrait $1 - \sigma^2/\mathrm{Var}(y)$, quantité qui varie d'un domaine à l'autre sans que le modèle change.

J'ai donc utilisé aussi une erreur normalisée par le niveau de bruit :

$$\frac{\mathrm{RMSE}}{\hat{\sigma}}$$

3.13.2 Pour la classification

RAINCOAT [11] recommande d'accompagner le taux de bonnes prédictions d'un F1 moyenné par classe, qui combine le rappel et la précision et traite les 6 activités à égalité quel que soit leur effectif. Je ne l'ai pas utilisé, je suis parti de l'accuracy, qui plafonnait autour de 0,9 en masquant les écarts, puis je suis passé au rappel par classe, qui les fait ressortir.

3.14 Les protocoles de validation

Sur les données de classification, la question posée est de savoir si le modèle généralise à des personnes nouvelles. Il faut donc que les sujets utilisés pour évaluer soient absents de l'entraînement. Une validation croisée ordinaire, qui mélangerait les fenêtres, laisserait un même sujet des deux côtés du découpage et ne répondrait plus à cette question. J'ai donc utilisé deux protocoles qui partitionnent par sujet :

- le GroupKFold, qui répartit les sujets en plusieurs groupes, chaque groupe servant de test à son tour ;
- le LeaveOneGroupOut, qui met un seul sujet de côté à chaque fois.

Les deux répondent à des questions différentes. Le second montre la variabilité entre sujets, le premier fournit une estimation plus stable de la performance, puisque chacun de ses plis moyenne plusieurs sujets.

La construction du jeu de données rend cette précaution nécessaire de façon très concrète. Les fenêtres sont extraites avec un recouvrement de 50 %, donc deux fenêtres consécutives d'un même sujet partagent exactement la moitié de leurs mesures brutes. Mélanger les fenêtres au hasard reviendrait à évaluer le modèle sur des données presque identiques à celles vues à l'apprentissage, et le score obtenu ne dirait plus rien sur une personne nouvelle.

3.15 Outils logiciels

J'ai travaillé en Python, dans des notebooks Jupyter, avec les bibliothèques suivantes :
— `numpy` et `pandas` pour la manipulation des tableaux et des séries indexées par le temps ;
— `matplotlib` pour les visualisations ;
— `scikit-learn` [6] pour les modèles de référence, la régression linéaire, la régression logistique et les validations croisées par groupe ;
— `statsmodels` [5] pour la décomposition STL et MSTL et les modèles ARIMA ;
— `adapt` pour les méthodes d'adaptation de domaine ;
— `scipy` pour le périodogramme.
Les notebooks ont été stockés sur GitHub tout au long du stage.

4 Travaux réalisés

4.1 La fabrication les décalages

4.1.1 Le générateur

J'ai écrit un générateur produisant des séries de la forme

$$x_t = \alpha\, t \;+\; A\,\sin\!\left(\frac{2\pi t}{P}\right) \;+\; \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2),$$

avec α la pente, A l'amplitude, P la période et σ le niveau de bruit. Une variante génère un bruit autocorrélé, $\varepsilon_t = \phi\,\varepsilon_{t-1} + e_t$, afin de disposer de séries dont le résidu n'est pas blanc.

Pour chacun des 5 décalages de la figure 3, je produis une paire de domaines en modifiant un seul paramètre, à 3 intensités croissantes. J'ai ajouté un sixième cas où les 5 décalages sont appliqués simultanément, pour se rapprocher d'une situation réelle.

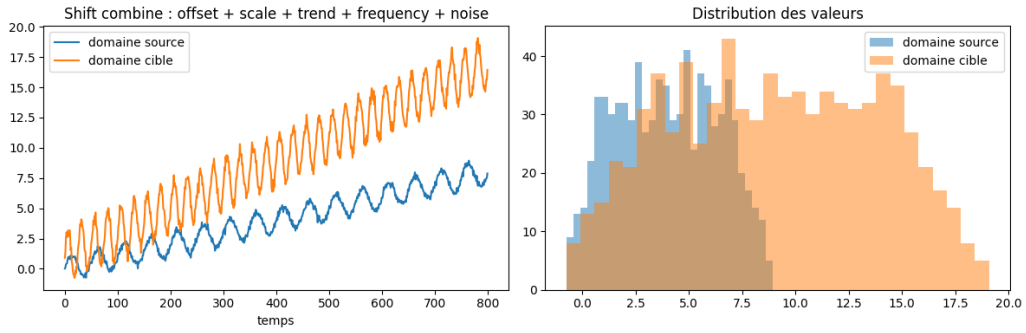


FIGURE 7 – Exemple de paire source-cible générée pour les 5 décalages.

4.1.2 Le pipeline de référence

J'ai d'abord mis en place un pipeline sans aucune adaptation : entraînement sur le domaine source seul, évaluation sur les deux domaines.

Le tableau montrait que le décalage de niveau ne dégradait presque rien.

En regardant la somme des coefficients de la régression, on obtient 0,9972, avec une ordonnée à l'origine de 0,022. Le modèle restituait donc la quasi-totalité du niveau qu'on lui donnait en entrée. Décaler toute la fenêtre d'une constante décalait la prédiction de la même constante.

J'en ai conclu qu'un modèle autorégressif absorbe naturellement un décalage de niveau, et qu'il n'y a rien à corriger de ce côté.

4.2 Le choix des jeux réels

J'ai recensé plusieurs jeux de données publics comportant un changement de domaine identifiable, en notant pour chacun la nature du décalage et la tâche associée. Le critère de sélection a été de couvrir 2 tâches différentes avec un décalage ayant un sens physique immédiat :

- une tâche de **prévision**, avec des relevés horaires de température sur plusieurs villes, le domaine étant la ville ;
- une tâche de **classification**, avec le jeu UCI HAR de reconnaissance d'activité humaine, le domaine étant le sujet portant le capteur.

Deux villes n'ont pas le même climat, deux personnes ne marchent pas de la même façon : dans les deux cas on peut interpréter un décalage.

4.3 La prévision de température

4.3.1 Les données

Le jeu contient la température horaire de trente-six villes sur 5 ans. J'ai retenu 4 villes de la côte ouest américaine comme domaine source, Seattle, Portland, San Francisco et Los Angeles, et une ville cible. Le domaine source est constitué en empilant les fenêtres des 4 villes.

Le choix de la ville cible a évolué. J'ai d'abord travaillé avec Jerusalem, avant de m'apercevoir que sa série contenait de longues plages de valeurs figées en plein milieu de la période d'apprentissage : 1 350 heures bloquées à la même valeur exacte entre novembre 2013 et janvier 2014, soit environ 4 % de la série. Ces valeurs ne sont pas des données manquantes signalées comme telles, mais une valeur par défaut recopiée dans le fichier, ce qui les rend invisibles à tout contrôle automatique.

Je l'ai donc remplacée par Miami, qui ne comporte aucune plage figée de cette ampleur et qui présente de plus un décalage plus marqué par rapport aux villes source.

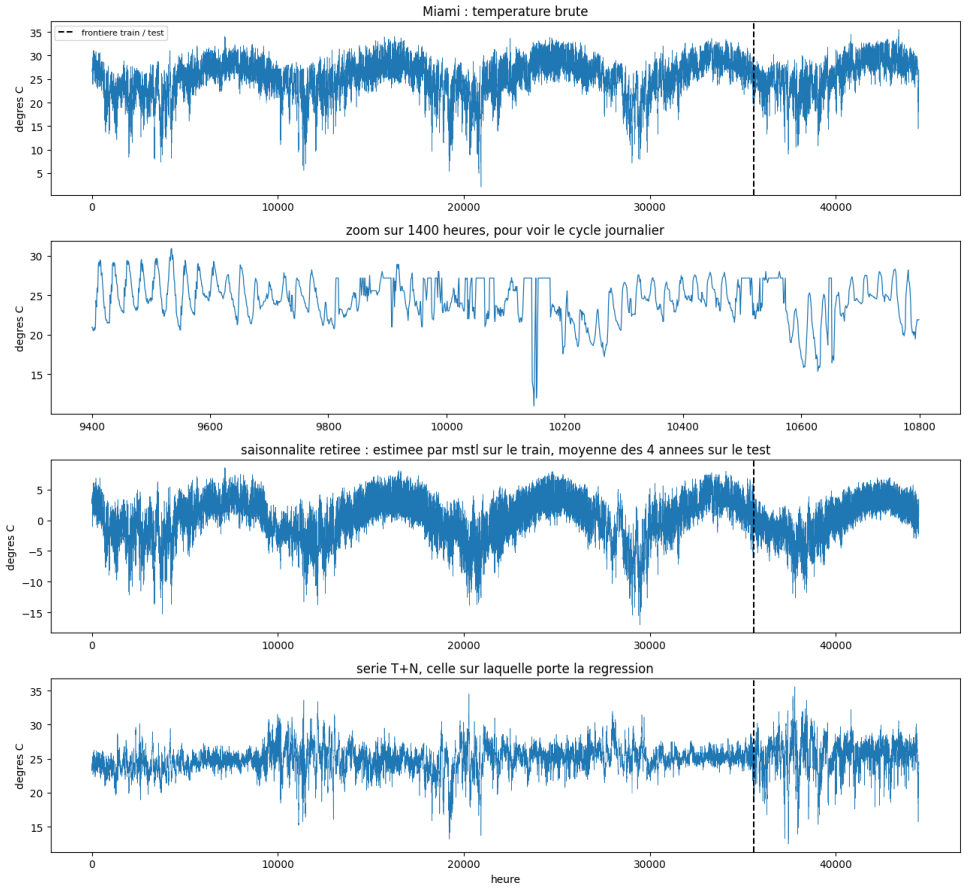


FIGURE 8 – Le signal de la ville cible (Miami) : température brute, zoom sur quelques jours, saisonnalité retirée, et série désaisonnalisée obtenue.

4.3.2 Deux versions du problème

Je compare systématiquement deux versions : la température brute, et la série désaisonnalisée $z_t = x_t - s_t$ obtenue en retirant les deux saisonnalités, journalière et annuelle.

Le cycle journalier est presque identique d'une ville à l'autre et domine la variance. Tant qu'il est présent, il masque toute différence de climat entre domaines. En le retirant, on force le modèle à apprendre ce qui varie d'une ville à l'autre, seul endroit où un changement de domaine peut se voir.

4.3.3 Retirer la saisonnalité sans tricher

La décomposition est ajustée sur les seuls 80 % d'apprentissage, faute de quoi la saisonnalité retirée du test serait estimée en utilisant le test lui-même. Sur la période de test, il n'y a donc plus rien d'estimé, et la saisonnalité doit être fabriquée.

J'ai d'abord recopié la dernière année d'apprentissage, avant de retenir la moyenne des 4 années complètes disponibles, position calendaire par position calendaire :

$$\bar{s}[j] \;=\; \frac{1}{4}\sum_{k=0}^{3} s\big[\, n_{\text{app}} - (k+1)\cdot 8760 + j \,\big], \qquad 0 \le j < 8760.$$

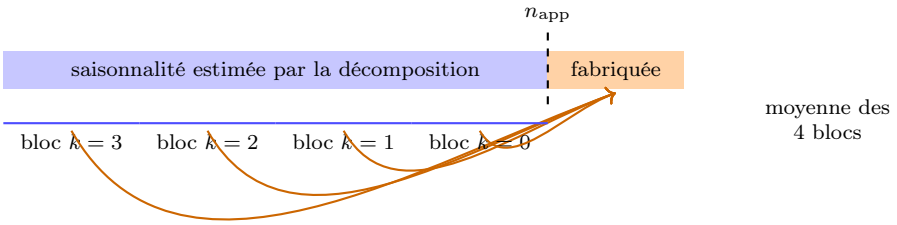


FIGURE 9 – Le prolongement de la saisonnalité. Les 4 blocs d'un an prélevés à la fin de l'apprentissage sont moyennés position par position, et le profil obtenu est répété sur le test.

Le bloc vaut 8760 heures et non 8766, bien que la période demandée à la décomposition soit 8766. La raison est que le bloc répété doit être un multiple de 24 pour retomber sur la même heure de la journée ; avec 8766, chaque répétition décale de six heures, soit un quart du cycle journalier.

La contrepartie est que 365 jours n'est pas une année réelle (365,25) : remonter d'un bloc recule d'environ un quart de jour dans la saison, et la dérive cumulée sur 4 blocs atteint une journée. L'alignement est donc exact à l'heure près pour le cycle journalier, et à un jour près pour le cycle annuel.

4.3.4 Un défaut de données

J'ai trouvé 4 défauts dans le fichier source.

La première ligne du fichier est vide pour toutes les villes. Les 26 heures suivantes forment une droite parfaite, décroissant de 0,002 degré par heure, avant que la série ne se mette à varier normalement. Ces valeurs ne sont pas une mesure : elles ont été fabriquées par le fournisseur des données pour combler le début. L'ampleur est faible, 26 heures sur 44 460, mais ces heures tombent précisément dans la fenêtre de visualisation au début du notebook.

Ensuite, le chargement des séries appliquait une interpolation puis supprimait les valeurs manquantes. Or l'interpolation de `pandas`, pour des valeurs manquantes situées après la dernière valeur valide, ne peut pas interpoler : elle prolonge la dernière valeur connue. Ces heures cessaient donc d'être manquantes et n'étaient plus supprimées.

Le fichier contient 792 heures manquantes en fin de série, pour certaines villes seulement. 8,8% de la période de test de la ville cible était donc une ligne plate.

J'ai corrigé le chargement et tronqué toutes les villes à une date commune afin de garantir un alignement identique. La série passe de 45 252 à 44 460 points par ville.

Enfin, en regardant les 5 villes ensemble, j'ai constaté que la température reste parfois strictement constante plusieurs heures d'affilée. Le phénomène touche toutes les villes.

ville	heures constantes	part	plages de 6 h ou plus	plus longue
San Francisco	1 741	3,92 %	14	43 h
Los Angeles	1 831	4,12 %	16	43 h
Seattle	2 616	5,88 %	20	43 h
Portland	3 224	7,25 %	28	48 h
Miami	4 420	9,94 %	33	43 h

Les plages les plus longues sont partagées. Celle de 43 heures qui commence le 18 juillet 2016 à 23 h apparaît dans les 5 villes au même instant, et celle de 33 heures du 18 janvier 2014 également. Il ne s'agit donc pas d'un défaut propre à une ville, mais de pannes de collecte comblées par le fournisseur sur l'ensemble du jeu.

Contrairement aux 3 défauts précédents, je n'ai rien corrigé à la suite de ce constat, et il ne fausse aucune mesure : ces heures sont des valeurs plausibles.

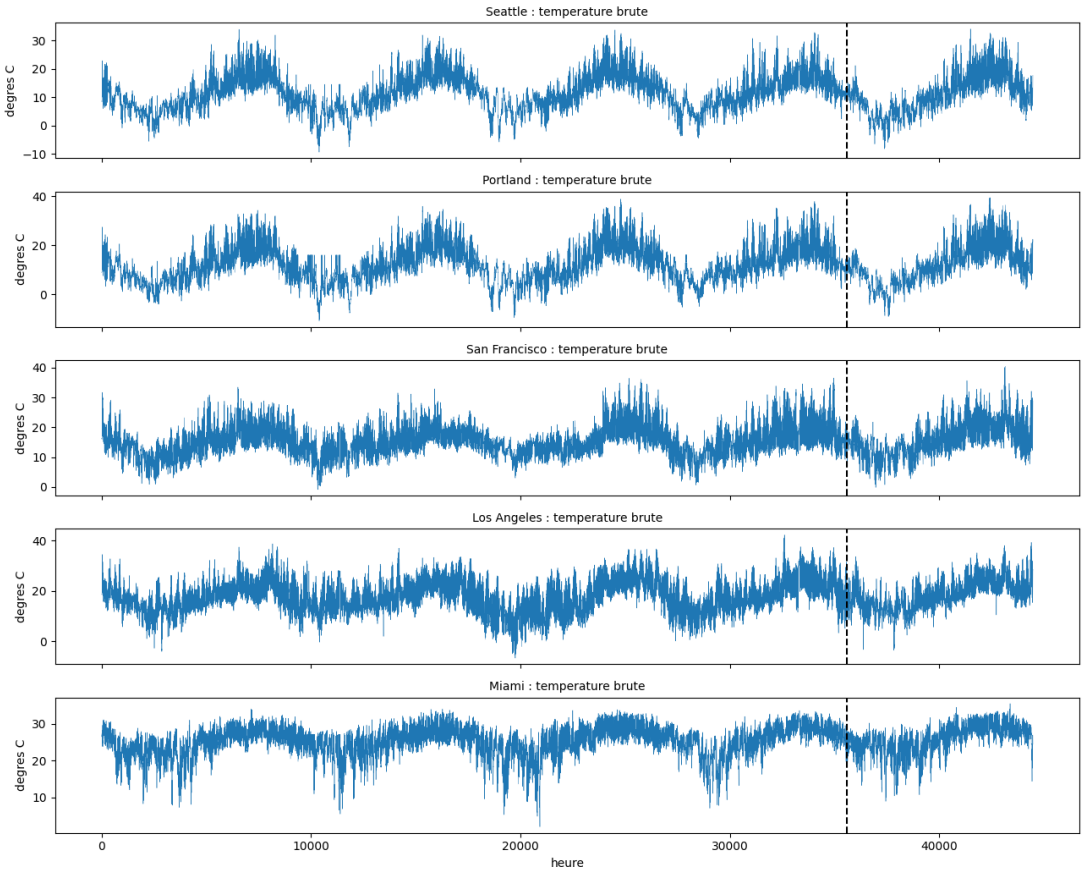


FIGURE 10 – Température brute pour les 4 villes source et la ville cible.

4.3.5 Les modèles comparés

Au modèle de référence, j'ai ajouté une famille de modèles ARIMA. Chacun est ajusté sur la période d'apprentissage, puis utilisé depuis chaque origine du test pour produire une prévision à 24 pas, la bibliothèque se chargeant d'itérer la récursion. J'ai également traité le transfert d'un modèle d'une ville à l'autre, en conservant les coefficients estimés sur la ville source et en les appliquant aux observations de la ville cible.

4.4 La classification d'activité

4.4.1 Les données

Le jeu UCI HAR [3] contient des enregistrements d'accéléromètre et de gyroscope de smartphone porté à la ceinture, pour 30 sujets réalisant 6 activités : marcher, monter un escalier, descendre un escalier, être assis, être debout, être couché. Chaque fenêtre de 2,56 secondes est décrite par 561 variables extraites des signaux temporels et fréquentiels. 21 sujets forment le domaine source, 9 le domaine cible.

Les capteurs échantillonnent à 50 Hz. Le signal est découpé en fenêtres de 2,56 secondes, soit 128 mesures, avec un recouvrement de 50 %, ce qui produit une nouvelle fenêtre toutes les 1,28 seconde. Une ligne du fichier est donc un résumé de 2,56 secondes.

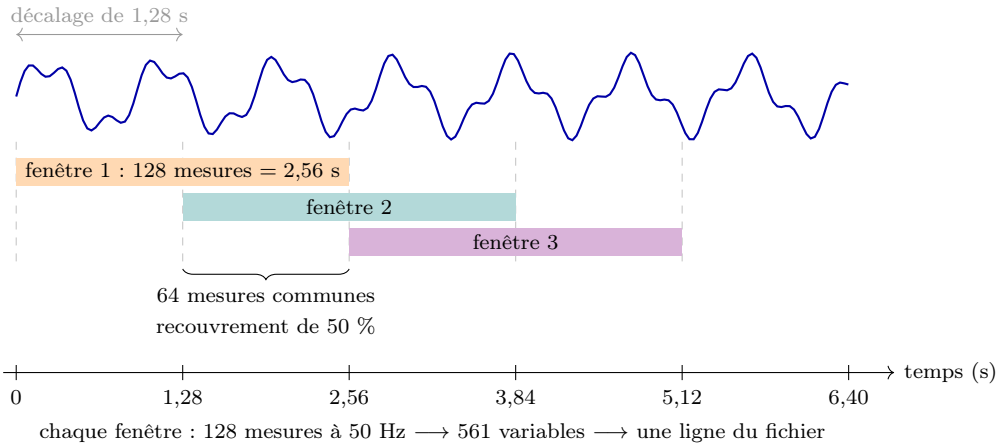


FIGURE 11 – Le découpage du signal dans UCI HAR.

Les 561 colonnes. La construction se fait en 3 temps. Les 2 capteurs donnent 6 axes bruts, 3 pour l'accéléromètre et 3 pour le gyroscope. Le filtrage, qui sépare l'accélération du corps de la gravité, la dérivation, qui donne les à-coups, la norme euclidienne et la transformée de Fourier en tirent 17 grandeurs, dont 10 temporelles et 7 fréquentielles. Parmi ces grandeurs, 8 sont triaxiales et valent donc 3 canaux chacune, les 9 autres sont des normes et n'en valent qu'un, soit $8 \times 3 + 9 = 33$ canaux. Enfin 17 statistiques produisent les 561 colonnes.

statistique calculée sur la fenêtre	colonnes	part
énergie par bande de fréquence	126	22,5 %
coefficients autorégressifs de Burg, ordre 4	80	14,3 %
moyenne	33	5,9 %
écart-type	33	5,9 %
écart absolu médian	33	5,9 %
maximum, minimum	33 chacun	5,9 % chacun
énergie, écart interquartile, entropie	33 chacune	5,9 % chacune
aire du signal	17	3,0 %
corrélation entre axes	15	2,7 %
indice de fréquence dominante	13	2,3 %
fréquence moyenne, asymétrie, aplatissement	13 chacune	2,3 % chacune
angles	7	1,2 %

Par domaine, 265 colonnes proviennent du temporel, 289 du fréquentiel et 7 sont des angles.

Toutes sont déjà normalisées dans $[-1, 1]$ et aucune n'est manquante, ce qui permet de les envoyer telles quelles dans un modèle.

L'organisation des fichiers. Le jeu décrit chaque fenêtre à deux niveaux, dans des fichiers alignés ligne à ligne : la ligne i de chacun parle de la même fenêtre.

fichier	dimensions	une ligne contient
`X_train.txt`	7352×561	les 561 statistiques de la fenêtre
`y_train.txt`	7352	l'activité réalisée
`subject_train.txt`	7352	le sujet concerné
`Inertial Signals/`	7352×128	les 128 mesures brutes, un fichier par canal

Une ligne de `X_train.txt` n'est donc pas un signal mais un résumé. Le signal brut se trouve dans le dossier `Inertial Signals`, réparti sur 9 fichiers : accélération totale, accélération du corps et vitesse angulaire, chacune sur 3 axes. Je m'en sers pour les visualisations et pour comparer deux sujets.

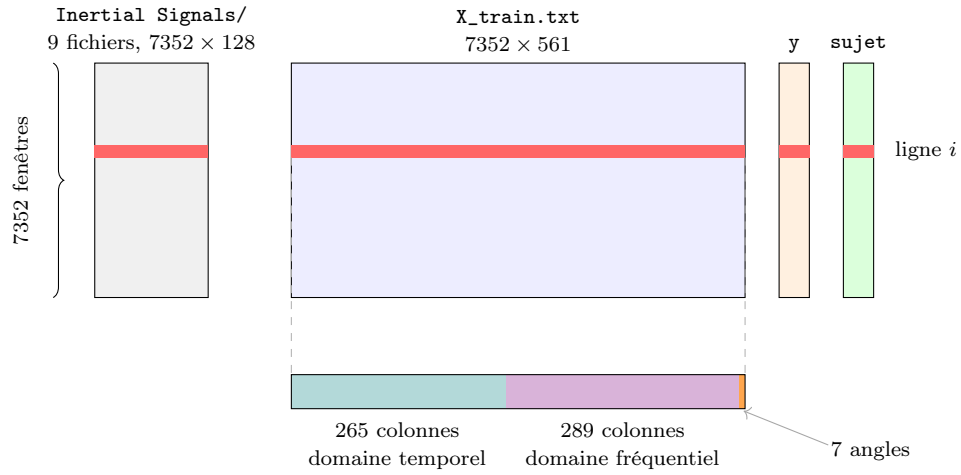


FIGURE 12 – La structure de `X_train.txt` et son alignement avec les autres fichiers. La ligne i décrit partout la même fenêtre : ses 128 mesures brutes dans `Inertial Signals`, ses 561 statistiques dans `X_train.txt`, son activité dans `y_train.txt` et la personne qui l'a produite dans `subject_train.txt`.

Chaque sujet consacre une à deux minutes à chaque activité. En temps cumulé, la marche dure 74 secondes en médiane, la descente d'escalier 61 et la position couchée 86, avec des écarts allant de 47 à 123 secondes selon les personnes. Un sujet passe ainsi 6 à 9 minutes au total sur le protocole, et le jeu complet représente 10 299 fenêtres, soit 3,7 heures de signal cumulé.

Le protocole est le même pour tout le monde. Chaque personne enchaîne les 6 activités dans un ordre fixe, debout, assis, couché, marche, descente puis montée d'escalier, et recommence une seconde fois. Le fichier reflète cet enchaînement : chaque sujet occupe un bloc de lignes contigu, et à l'intérieur de ce bloc les activités se succèdent dans l'ordre du protocole.

Comme deux fenêtres qui se suivent dans le temps partagent 64 de leurs 128 mesures, il suffit de comparer la seconde moitié d'une fenêtre à la première moitié de la suivante pour savoir si elles sont contiguës. Le test porte sur les 10 297 couples de lignes voisines des deux fichiers.

Dans 9 442 cas les 64 mesures coïncident, les deux fenêtres se suivent donc réellement dans le temps. Dans 855 cas elles diffèrent, soit une jonction sur douze. J'appelle tronçon un morceau d'enregistrement d'un seul tenant, entre deux de ces coupures. Le jeu en compte 857, répartis sur 400 blocs d'activité, soit un peu plus de 2 par bloc.

Le découpage en blocs et en tronçons n'est pas le même selon l'activité. Les valeurs ci-dessous sont des moyennes sur les 30 sujets. Elles sont très régulières pour les 3 postures, où presque chaque sujet a 2 blocs de 2 tronçons, et nettement plus dispersées pour les escaliers, où le nombre de blocs varie de 2 à 5 selon les personnes.

activité	blocs par sujet (moyenne)	tronçons par bloc (moyenne)
debout, assis, couché	2,0	2,0
marche	2,0	2,1
montée d'escalier	2,6	2,4
descente d'escalier	2,7	2,3

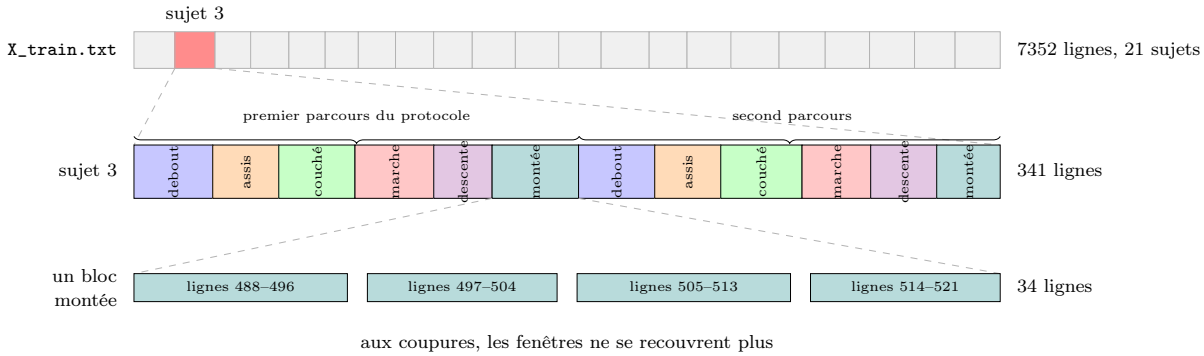


FIGURE 13 – L'ordre des lignes du fichier. En haut, les 7352 lignes d'apprentissage sont regroupées par sujet, chacun occupant un bloc contigu. Au milieu, les 341 lignes du sujet 3 se répartissent en 12 blocs d'activité, les 6 activités du protocole étant parcourues deux fois dans le même ordre. En bas, un bloc se décompose lui-même en tronçons, séparés par des coupures où le recouvrement de 50 % est interrompu.

Le découpage est ensuite confié à une fonction qui ne sélectionne jamais des lignes, mais des sujets. Elle construit des masques booléens sur le vecteur des sujets : les 4 derniers sujets du domaine source servent à évaluer sur ce domaine, et le domaine cible est coupé en 6 sujets pour l'entraînement et 3 pour l'évaluation. Aucune personne n'apparaît des deux côtés d'une frontière.

Le décalage de domaine est ici le sujet. Deux personnes ne marchent pas de la même façon et ne portent pas le téléphone exactement de la même manière.

Au début je pensais appliquer à ce jeu le retrait de saisonnalité utilisé en météo. Or en météo, le cycle journalier est commun à toutes les villes et masque le décalage : le retirer révèle ce qui distingue les villes. En classification d'activité, la périodicité est au contraire l'information de classe, puisque la marche est périodique et l'immobilité ne l'est pas. La retirer effacerait ce qui distingue les activités.

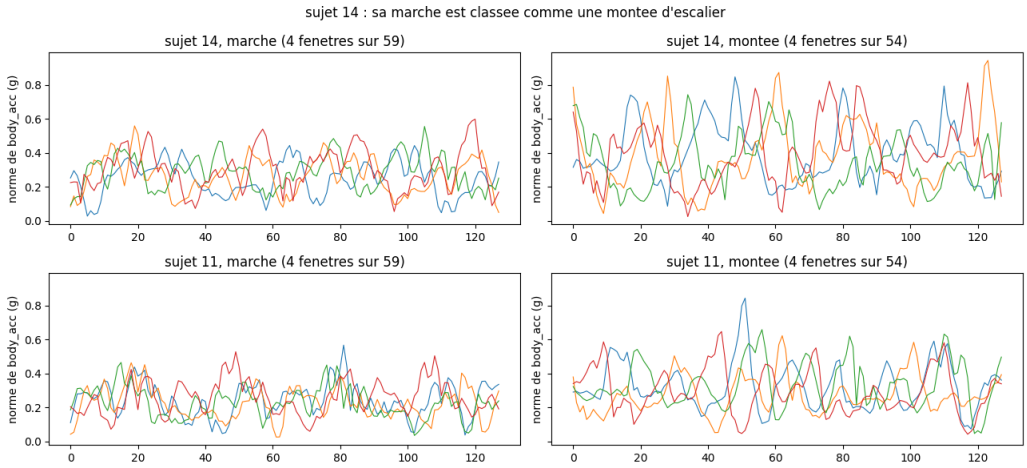


FIGURE 14 – Signaux bruts pour deux activités comparés entre un sujet difficile à classer et un sujet facile.

Le découpage officiel donne 21 sujets source contre 9 cible. Une partie de l'avantage du modèle source vient donc de la quantité de données et non du domaine. J'ai construit une variante équilibrée à 9 sujets de chaque côté, tirés aléatoirement, avec une moyenne sur plusieurs tirages afin de ne pas dépendre d'un tirage particulier.

4.4.2 Chercher où le modèle se trompe

J'ai produit successivement la matrice de confusion, le rappel par classe, puis le rappel par couple sujet-classe. J'ai également examiné individuellement les sujets les plus mal classés, en revenant aux signaux inertiels bruts.

Pendant une activité statique, l'accélération du corps est nulle. L'accéléromètre ne mesure donc que la gravité, dont la norme vaut un dans l'unité du jeu de données. Vérifier cette norme sujet par sujet permet de distinguer un capteur défaillant d'un comportement inhabituel.

4.4.3 Scénario réaliste

Un nouveau sujet arrive et l'on ne dispose que de deux ou trois enregistrements par activité. On adapte le modèle source sur ces quelques fenêtres et on évalue sur tout le reste de ses données.

J'ai fait varier trois choses : le paramètre λ de rappel vers la source, le nombre d'activités disponibles pour l'adaptation, et la qualité du modèle source de départ, en comparant un modèle appris sur les 21 sujets à un modèle appris sur deux seulement.

4.5 Organisation du code

Le travail est organisé autour de 2 modules partagés entre tous les notebooks.

module	contenu
`adapt_utils.py`	découpage en fenêtres, découpage par sujet, entraînement et évaluation, comparaison des méthodes d'ADAPT, courbes d'apprentissage
`series_utils.py`	génération de séries synthétiques, estimation de période, mesures d'erreur

5 Résultats

5.1 Sur données synthétiques

5.1.1 Vérifier l'extraction avant de s'en servir

Un notebook génère des séries dont la pente, la période, l'amplitude saisonnière et le niveau de bruit sont choisis, puis les réestime par décomposition STL, en faisant varier chaque paramètre un à un.

Deux paramètres sur quatre sont retrouvés exactement. La pente est estimée sans erreur mesurable sur toute la plage testée, et la période l'est aussi, à moins de 1 % près sur quelques valeurs. Les deux autres échouent : l'amplitude et le bruit.

J'ai ensuite vérifié le cas à deux périodes, qui est celui des données réelles. Sur une série construite avec un cycle de 24 heures et un cycle de 168 heures, le périodogramme retrouve exactement ces 2 valeurs, et la décomposition MSTL sépare les deux saisonnalités. Ce test reste toutefois plus facile que le cas réel : les 2 périodes y sont dans un rapport de 7, alors que le cycle journalier et le cycle annuel de la température sont dans un rapport de 365. Sur les données réelles, le périodogramme ne sépare pas le cycle annuel avec assez de finesse, et je lui fournis directement sa valeur.

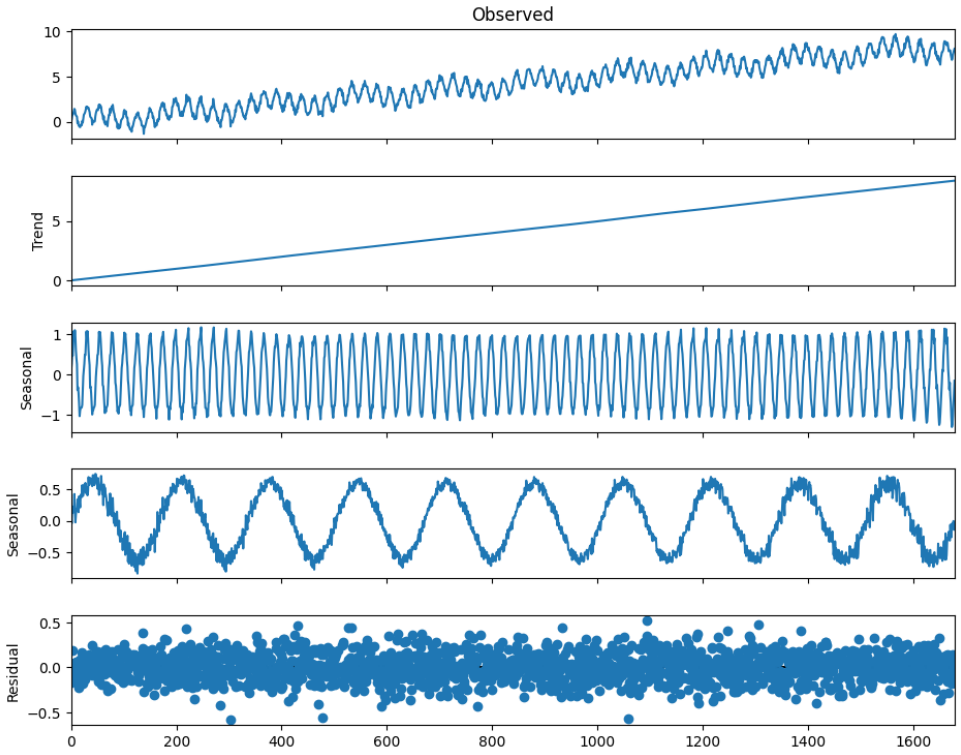


FIGURE 15 – Décomposition MSTL d'une série à deux périodes, 24 et 168 heures, détectées automatiquement par le périodogramme.

5.1.2 Tous les décalages ne se valent pas

Le modèle de référence obtient un R^2 de 0,951 sur le domaine source. Le tableau suivant donne ce qu'il devient sur le domaine cible, sans aucune adaptation.

décalage	intensité 0,25	0,50	1,00
niveau	0,944	0,944	0,944
échelle	0,961	0,971	0,983
tendance	0,951	0,957	0,968
bruit	0,915	0,881	0,800
fréquence	0,940	0,909	0,621
combiné	0,940	0,908	0,619

Les 3 premiers décalages sont des transformations affines de la série : ajouter une constante, ajouter une pente, multiplier une composante par un facteur, ils ne coûtent rien et le score augmente même avec l'intensité pour l'échelle et la tendance. Ce n'est pas que la prédiction s'améliore : le R^2 se mesure relativement à la variance de la cible, et ces décalages augmentent cette variance, ce qui modifie le rapport à erreur inchangée.

Le décalage de bruit se lit à part. Le générateur fixe l'écart-type du bruit cible à $\sigma_{\mathcal{T}} = \sigma_{\mathcal{S}} \times (1+i)$, où i est l'intensité, donc il passe de 0,15 à 0,30 dans la dernière colonne. Or ce bruit est imprévisible par construction : en ajouter abaisse le score que le meilleur modèle possible pourrait atteindre, quel que soit ce modèle. Une partie de la baisse de 0,951 à 0,800 était donc inévitable, et ne traduit pas un échec du transfert.

C'est sur le décalage de fréquence, où le score tombe à 0,621, que l'adaptation a une chance de servir. Le décalage combiné descend aussi bas, à 0,619, mais il mélange 5 effets et se prête donc moins bien à l'analyse.

5.1.3 9 méthodes

Sur ce décalage de fréquence :

méthode	intensité 0,25	0,50	1,00
sans adaptation	0,940	0,909	0,621
FA	0,950	0,957	0,961
RegularTransferLR	0,950	0,957	0,961
KLIEP	0,938	0,917	0,712
TrAdaBoostR2	0,920	0,931	0,931
KMM	0,913	0,874	0,495
fMMD	0,885	0,866	−0,074
SA	−0,123	−0,069	−0,404
CORAL	−2,274	−1,941	−2,103
TCA	−4,679	−8,001	−4,623

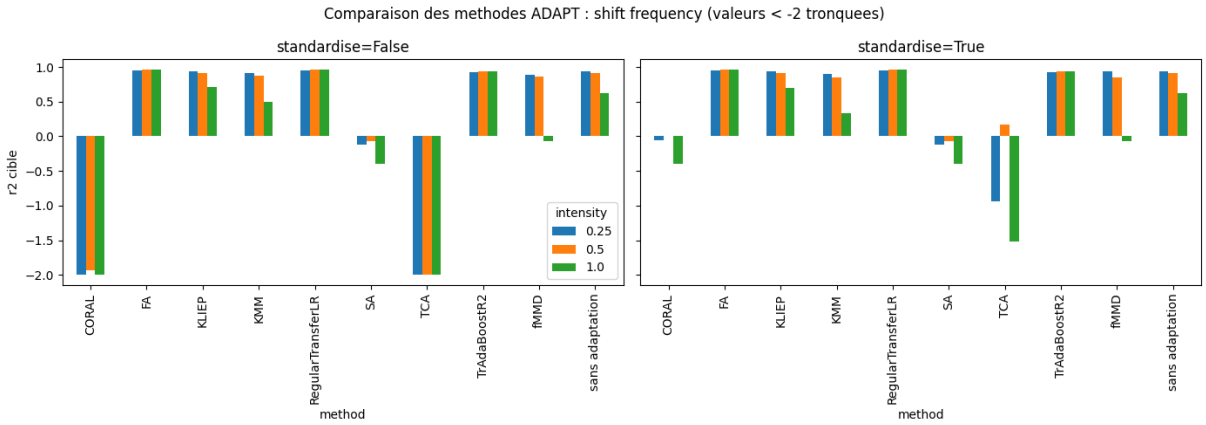


FIGURE 16 – Comparaison des 9 méthodes sur le décalage de fréquence, aux 3 intensités.

3 méthodes seulement améliorent nettement. FA et RegularTransferLR font passer le R^2 de 0,621 à 0,961 à l'intensité maximale, et TrAdaBoostR2 à 0,931. La suivante, KLIEP, est très loin derrière avec 0,712. Toutes trois utilisent les étiquettes du domaine cible, ce qui explique en grande partie leur avantage et doit être rappelé dans toute comparaison : aucune méthode non supervisée n'y parvient.

3 méthodes dégradent systématiquement et massivement. CORAL, SA et TCA produisent des R^2 négatifs, c'est-à-dire pires que prédire une constante.

5.1.4 Un modèle qui se rabat sur la moyenne qu'il a apprise

Avant d'en arriver aux décalages entre domaines, une expérience sur série synthétique m'a fait rencontrer un biais que je retrouverai plus loin sur les données réelles. Un ARIMA ajusté avec $d = 0$ suppose une moyenne fixe. Sur une série qui monte, il ramène donc sa prévision vers la moyenne de sa période d'apprentissage. Cette moyenne vaut 1,901 alors que le niveau atteint en fin d'apprentissage est de 3,746, soit un écart de 1,845. Le biais mesuré sur la prévision est de $-2{,}034$, et l'erreur normalisée de 2,443. En différenciant, avec $d = 1$, le biais tombe à $-0{,}274$ et l'erreur à 1,359.

Le balayage de 11 ordres confirme que la différenciation compte davantage que l'ordre : tous les modèles à $d = 0$ se tiennent autour de 2,4 avec un biais voisin de -2, et tous ceux à $d = 1$ autour de 1,34 avec un biais proche de zéro.

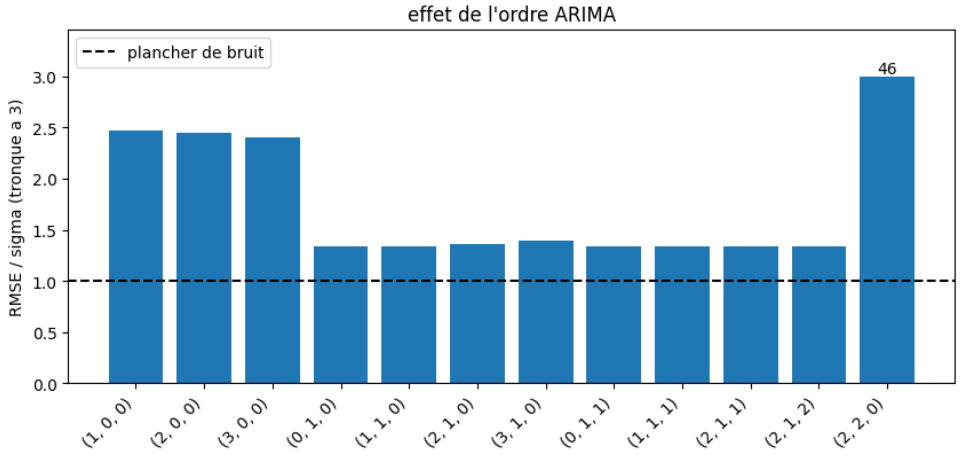


FIGURE 17 – Erreur et biais des 11 ordres testés.

Ce résultat annonce celui de la partie météo. Dans les deux cas, un modèle se rabat sur la moyenne qu'il a apprise, et le biais vaut à peu près l'écart entre cette moyenne et le niveau réel. Seule la cause de l'écart change : le temps ici, le changement de domaine là-bas.

5.1.5 Un sixième décalage : la mémoire du bruit

En faisant varier un à un les paramètres de la série synthétique, puis en transférant les coefficients autorégressifs d'un domaine à l'autre, j'ai obtenu un écart nul entre le modèle transféré et un modèle réajusté sur la cible, dans presque tous les cas.

paramètre modifié	valeurs	RMSE	écart
pente	0 à 0,02	1,462 à 2,057	0,000
amplitude saisonnière	1 à 20	1,435	0,000
période saisonnière	12 et 24	1,435	0,000
période saisonnière	48 et 168	5,6	0,088 et 0,003
écart-type du bruit	0,5 à 4	1,414 à 1,454	0,000

La RMSE dit si la tâche est difficile, l'écart dit ce que coûte le transfert. C'est le second qui nous intéresse, et il est nul presque partout, chaque fois pour une raison différente.

L'amplitude saisonnière ne change rien parce que STL retire la saisonnalité avant l'ARIMA : la série désaisonnalisée est identique, et la RMSE l'est aussi au millième près. La pente disparaît à la différenciation, donc les deux modèles apprennent la même relation, même lorsque la tâche devient plus difficile. La période n'est un problème que lorsqu'elle cesse d'être compatible avec celle donnée à STL, fixée à 24 : un cycle de 12 heures est aussi 24-périodique et se retire correctement, un cycle de 48 ou 168 heures ne l'est pas, la saisonnalité résiduelle passe alors dans la série désaisonnalisée et fait augmenter l'erreur. C'est le seul cas où un écart apparaît. Le bruit enfin ne change rien parce qu'après différenciation il ne reste que lui : la série est proportionnelle à son bruit, et la multiplier par une constante ne touche pas aux coefficients, qui ne dépendent que des autocorrélations. Cela ne contredit pas le tableau précédent, où le décalage de bruit dégradait bien le score : sur la série brute, le bruit s'ajoute à une saisonnalité, et en augmenter l'écart-type réduit la part prévisible de la variance, donc l'autocorrélation. Doubler σ fait ainsi passer l'autocorrélation d'ordre 1 de 0,992 à 0,980. Une fois la saisonnalité retirée et la série différenciée, il ne reste que le bruit. Je l'ai vérifié directement : en multipliant l'écart-type du bruit par 100, l'estimation de φ_1 reste identique à 0,9035, alors que la variance estimée est multipliée par 10 010.

Aucun des paramètres du générateur ne modifie donc la relation entre la fenêtre et la valeur à prédire. Pour y parvenir, il faut changer la mémoire du bruit elle-même.

J'ai donc construit un sixième cas, où la série est produite en faisant passer un bruit blanc dans une récursion autorégressive :

$$u_t = \varphi\, u_{t-1} + \varepsilon_t,$$

ce qui donne au bruit une mémoire réglée par φ. Un φ proche de 1 produit un bruit très corrélé dans le temps, un φ nul redonne un bruit blanc ordinaire. Cette forme n'est pas arbitraire. C'est le modèle d'erreur autorégressive d'ordre 1 employé dans l'article [10], qui l'écrit $\epsilon(t) = \rho\,\epsilon(t-1) + \nu(t)$ avec ν un bruit blanc.

Le même article donne la situation physique correspondante. Un bruit à mémoire apparaît quand l'appareil de mesure réagit lentement aux changements du système : l'écart entre la vraie valeur et la valeur relevée persiste d'un instant au suivant, et deux relevés proches dans le temps se ressemblent donc plus qu'ils ne le devraient. L'article mentionne des exemples avec des capteurs dans des situations réelles d'électrophysiologie cardiaque ou d'électrochimie. L'hypothèse habituelle d'un bruit indépendant est alors fausse, et la supposer à tort conduit à sous-estimer l'incertitude sur les paramètres.

J'ai entraîné un modèle sur une source de mémoire $\varphi = 0{,}9$, puis je l'ai transféré vers des cibles de mémoire décroissante, en comparant à chaque fois avec un modèle réajusté sur la cible.

φ de la cible	modèle transféré	modèle réajusté	écart
0,9	0,8516	0,8507	0,11 %
0,6	0,9290	0,8976	3,50 %
0,3	1,0378	0,9397	10,44 %
0,0	1,1941	0,9690	23,23 %

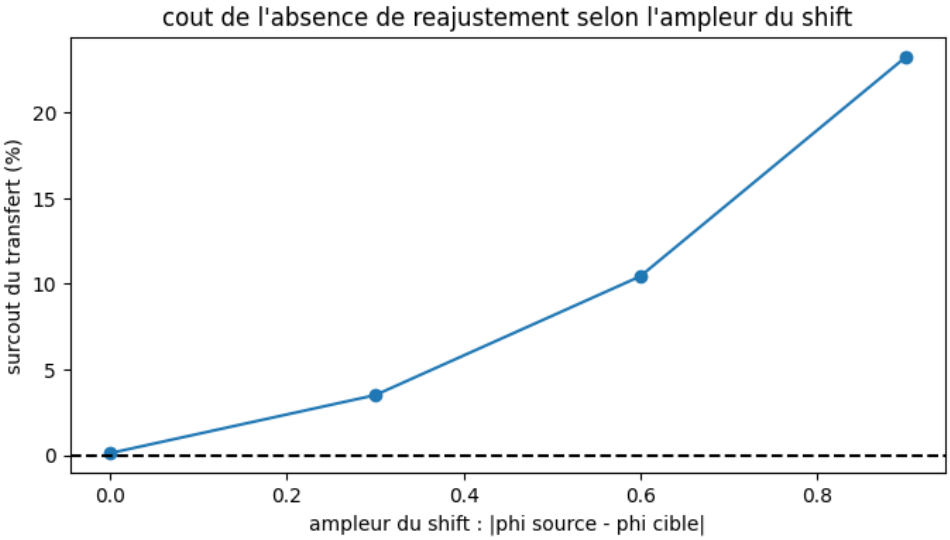


FIGURE 18 – Transfert d'un modèle appris sur un bruit de mémoire $\varphi = 0{,}9$ vers des cibles de mémoire décroissante.

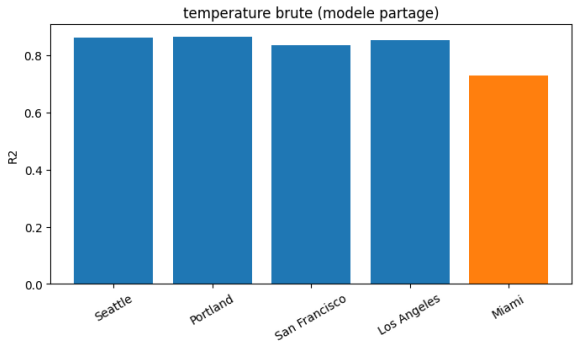
Le coût du transfert croît régulièrement avec l'écart de mémoire, de 0,11 % quand les deux domaines ont la même mémoire à 23 % quand la cible n'en a plus aucune. C'est cohérent avec ce que montre le tableau précédent : ce qui coûte cher n'est pas de déplacer la série, mais de changer la façon dont elle évolue.

5.2 Sur la prévision de température

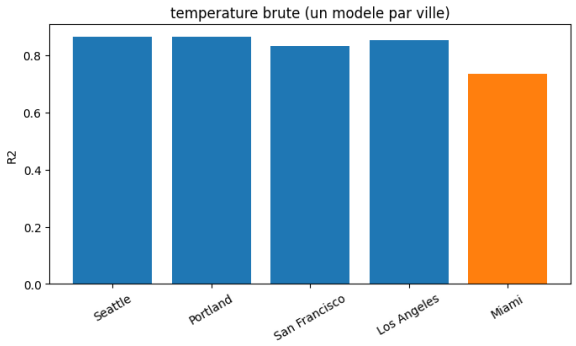
5.2.1 Le décalage n'apparaît qu'après désaisonnalisation

R^2	source → source	source → cible	cible → cible
température brute	0,857	0,730	0,734
série désaisonnalisée	0,485	0,284	0,306

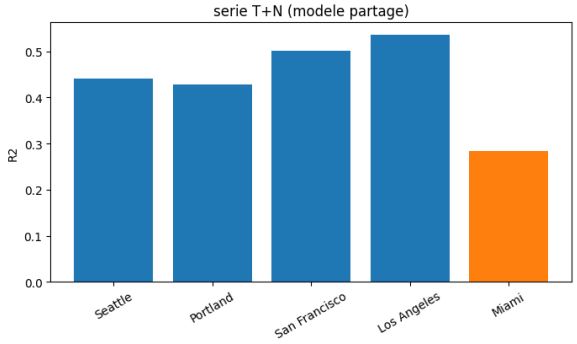
Sur la température brute, le transfert atteint presque le modèle appris sur place. Le cycle journalier, commun à toutes les villes, suffit à prédire et masque la différence de climat. Sur la série désaisonnalisée, un écart apparaît.



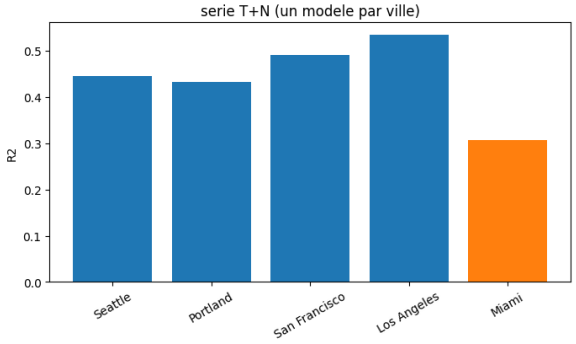
série brute, modèle partagé



série brute, un modèle par ville



série T+N, modèle partagé



série T+N, un modèle par ville

FIGURE 19 – R^2 par ville. D'abord la température brute, ensuite la série désaisonnalisée. À gauche un modèle unique appris sur les 4 villes source, à droite un modèle par ville.

5.2.2 Ce que mesure vraiment l'erreur normalisée

Le rapport RMSE/$\hat{\sigma}$ était censé permettre de comparer les villes entre elles.

Au numérateur. Sur la période de test, la saisonnalité retirée est fabriquée et non estimée. La cible du modèle contient donc un terme supplémentaire, absent de la période d'apprentissage :

$$z_t \;=\; \tau_t \;+\; r_t \;+\; \underbrace{\left(s_t - \bar{s}_t\right)}_{\text{erreur d'extrapolation}} \quad .$$

J'ai mesuré ce terme en remplaçant la saisonnalité fabriquée par une décomposition ayant vu le test, qui joue ici le rôle de référence inatteignable. Le tableau compare, sur la période de test, l'écart-type de la série désaisonnalisée dans les 2 cas.

ville	saisonnalité fabriquée	référence inatteignable
Seattle	3,54	2,01
Portland	4,01	2,31
San Francisco	3,40	1,78
Los Angeles	3,60	2,19
Miami, la ville cible	2,54	1,60

La série que le modèle doit prédire est donc nettement plus dispersée qu'elle ne le serait sans ce défaut. Cette dispersion supplémentaire ne vient pas de la météo, mais de la façon dont la saisonnalité a été prolongée sur le test.

L'effet pèse sur toutes les villes, mais pas également : le rapport entre les 2 colonnes va de 1,59 pour Miami à 1,91 pour San Francisco.

Au dénominateur. Le $\hat{\sigma}$ était estimé sur l'apprentissage alors que l'erreur porte sur le test. Le rapport entre les deux varie d'une ville à l'autre :

ville	$\hat{\sigma}_{\text{app}}$	$\hat{\sigma}_{\text{test}}$	rapport
Seattle	2,25	2,00	1,13
Portland	2,71	2,28	1,19
San Francisco	1,98	1,75	1,13
Los Angeles	2,38	2,19	1,09
Miami	1,77	1,59	1,11

J'ai retenu que la RMSE utilise de toute façon les étiquettes de test, et qu'il n'y a donc aucune raison de s'interdire le $\hat{\sigma}$ du test. J'ai adopté $\hat{\sigma}_{\text{test}}$.

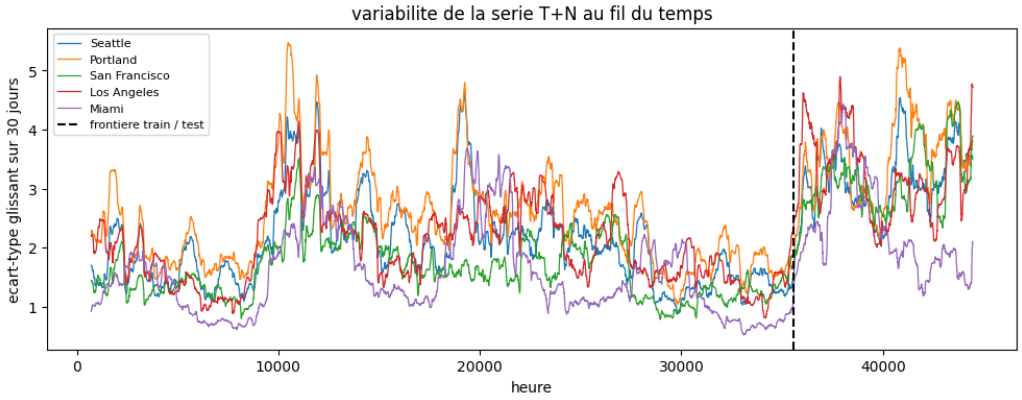


FIGURE 20 – Écart-type de la série désaisonnalisée. Le changement de niveau à la frontière apprentissage-test mesure le défaut d'extrapolation de la saisonnalité, non un changement réel de météo.

5.2.3 Un simple recentrage bat deux des 3 méthodes

Sur la série désaisonnalisée, le transfert vers Miami donne une erreur normalisée de 1,639, contre 1,328 pour un modèle appris sur place. C'est cet écart que j'ai cherché à expliquer, et non celui du R^2 : le coefficient de détermination rapporte l'erreur à la variance de la cible, qui n'est pas la même d'un domaine à l'autre, alors que l'erreur normalisée la rapporte au niveau de bruit et reste donc comparable.

En cherchant d'où venait cet écart, j'ai mesuré la somme des coefficients de la régression : 0,8823. Le modèle ne restitue donc que 88 % du niveau qu'on lui donne en entrée, et remplace le reste par une constante calée sur la moyenne de son domaine d'apprentissage.

Ce comportement n'a rien d'un défaut d'implémentation : c'est exactement ce dont on parlait plus haut. Pour un processus autorégressif, la prévision à h pas vaut $\varphi^h \eta_t$, et ce facteur s'écrase quand l'horizon s'allonge. Le prédicteur optimal abandonne progressivement l'information du présent pour se rabattre sur le niveau moyen : la variance de l'erreur de prévision croît avec l'horizon jusqu'à atteindre la variance du processus.

Ce comportement devient nuisible dès qu'on change de domaine, car la moyenne vers laquelle le modèle se rabat est celle de son apprentissage. Le modèle prend une journée ordinaire à Miami pour une journée anormalement chaude. Il prédit alors un retour à la normale qui n'arrive pas.

L'ampleur du phénomène se prédit exactement. Avec Δ l'écart de niveau entre les deux domaines, le biais attendu vaut

$$-\Big(1-\sum_j a_j\Big)\,\Delta.$$

quantité	valeur
écart de niveau entre Miami et la source	11,00
fraction non transmise, $1-0{,}8823$	0,1177
biais attendu	$-1{,}294$
biais mesuré	$-1{,}370$

La correction : retirer à chaque ville sa propre moyenne d'apprentissage avant de modéliser.

méthode	RMSE/$\hat{\sigma}_{\text{test}}$	utilise les étiquettes cible
sans adaptation, sans recentrage	1,639	non
CORAL	1,464	non
KMM	1,356	non
sans adaptation, avec recentrage	1,348	non
RegularTransferLR	1,328	oui
modèle appris sur la cible	1,328	oui

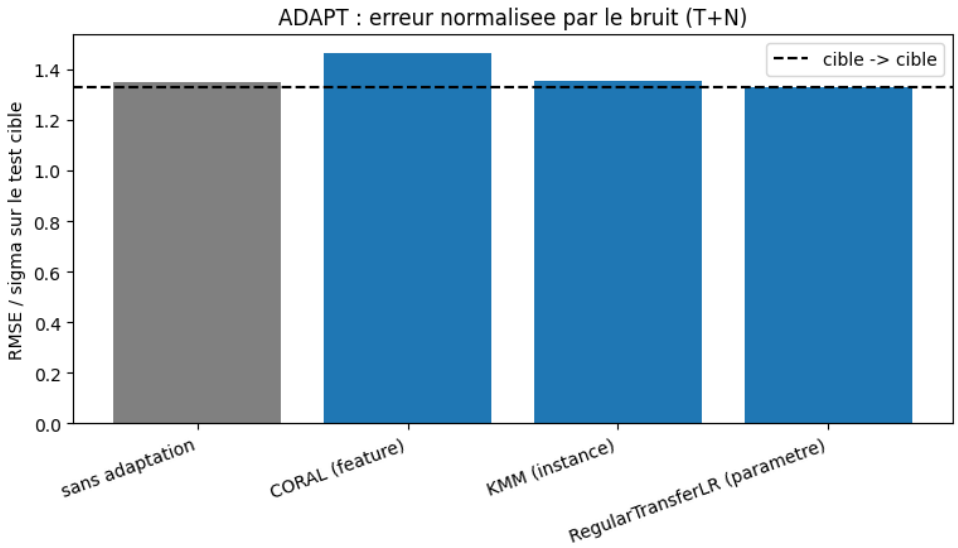


FIGURE 21 – Comparaison des méthodes d'adaptation sur la ville cible, série désaisonnalisée. La ligne horizontale marque le modèle appris directement sur la cible.

Le recentrage amène le transfert proche de celui appris sur place, sans utiliser une seule étiquette de la cible, là où les 2 méthodes non supervisées échouent à faire mieux que ne rien faire.

5.2.4 Travailler sur les variations

Le recentrage par ville corrige le biais en retirant une moyenne. Une autre façon de faire disparaître le niveau est de ne jamais le manipuler. J'ai donc construit une seconde représentation,

dans laquelle l'entrée devient la suite des variations d'une heure à l'autre à l'intérieur de la fenêtre, et la cible le changement total sur l'horizon :

$$X_i = (\Delta z_{i+1}, \ldots, \Delta z_{i+23}), \qquad y_i = z_{i+47} - z_{i+23}, \qquad \Delta z_t = z_t - z_{t-1}.$$

La prévision en niveau se retrouve en ajoutant z_{i+23}, dernière valeur de la fenêtre. Comme on ajoute la même quantité à la prévision et à la vérité, l'erreur est inchangée et les chiffres restent comparables aux précédents.

représentation	RMSE	RMSE/$\hat{\sigma}_{\text{test}}$	erreur moyenne	$\sum_j a_j$
niveau sans recentrage, source → cible	2,606	1,639	−1,370	0,882
niveau recentré, source → cible	2,144	1,348	−0,177	0,736
niveau recentré, cible → cible	2,111	1,328	−0,273	0,592
variations, source → cible	2,308	1,452	0,005	−2,591
variations, cible → cible	2,302	1,448	0,008	−4,211

La différenciation supprime complètement le biais et facilite le transfert vers de nouvelles données sans rien exiger d'elles. En contrepartie, elle réduit la performance du modèle.

5.2.5 Ce que vaut une prédiction sans modèle

Avant de comparer des modèles, il faut savoir ce que rapporte le fait de ne rien modéliser.

prédiction	RMSE/$\hat{\sigma}_{\text{test}}$
dernière valeur connue	1,486
moyenne de la fenêtre	1,598
moyenne du domaine d'apprentissage	1,650
régression sur 24 retards, apprise sur la cible	1,328

La régression gagne 11 % sur la meilleure de ces références. C'est un gain réel mais modeste, et il donne l'échelle à laquelle lire tous les écarts de cette partie.

Le transfert sans recentrage, à 1,639, fait à peine mieux que prédire la moyenne du domaine d'apprentissage, à 1,650. Un modèle qui se rabat sur sa moyenne finit par ne plus valoir mieux qu'elle.

5.2.6 Régression contre ARIMA

modèle	RMSE	RMSE/$\hat{\sigma}_{\text{test}}$	AIC
régression sur 24 retards	2,111	1,328	—
ARIMA$(2,0,0)$	2,183	1,373	58 239
ARIMA$(1,0,1)$	2,186	1,375	58 268
ARIMA$(2,0,1)$	2,191	1,378	58 031
ARIMA$(1,0,0)$	2,223	1,398	58 428
ARIMA$(3,0,0)$	2,229	1,402	57 993
ARIMA$(2,1,1)$	2,363	1,486	58 732

La régression directe l'emporte, malgré un désavantage apparent : elle ne voit que 24 valeurs quand l'ARIMA résume tout le passé dans son état.

L'AIC ne classe pas comme l'erreur. Sur ces 6 modèles, l'ordre $(3,0,0)$ a le meilleur AIC et la pire erreur de sa famille. Ce n'est pas une anomalie : l'AIC mesure la qualité de l'ajustement

à un pas, l'erreur porte sur 24.

La différenciation dégrade le modèle. L'ordre $(2, 1, 1)$ est le pire du tableau, ce qui confirme que la série désaisonnalisée est déjà stationnaire et que la différencier ajoute du bruit sans rien retirer. Son erreur, 2,363, est d'ailleurs celle de la simple répétition de la dernière valeur connue, 2,362. Le seul modèle différencié du tableau ne fait donc pas mieux que ne rien prédire du tout.

Ce dernier point est cohérent avec l'absence d'apport des termes de moyenne mobile, annoncée en section 3 : un MA(q) n'a aucune mémoire au-delà de q pas.

5.2.7 Jusqu'où remonter dans le passé

Le tableau précédent ne compare que 6 modèles choisis à la main. Pour savoir quel ordre autorégressif retenir, j'ai balayé p de 1 à 24 sur la ville cible, en mesurant à chaque fois l'erreur de prévision à 24 pas sur le test et l'AIC sur l'apprentissage.

Les prévisions de ce balayage portent sur une origine sur 8 au lieu de toutes, afin de tenir dans un temps de calcul raisonnable : 24 ordres sur toutes les origines représentent plus de 200 000 prévisions. Ce sous-échantillonnage a un défaut : les origines retenues tombent toujours sur les mêmes 3 heures de la journée, 6 h, 14 h et 22 h.

p	RMSE/$\hat{\sigma}_{\text{test}}$	AIC	p	RMSE/$\hat{\sigma}_{\text{test}}$	AIC
1	1,394	58 428	13	1,346	57 226
2	1,368	58 239	14	1,345	57 175
3	1,398	57 993	15	1,344	57 172
4	1,391	57 982	16	1,345	57 007
5	1,408	57 919	17	1,347	56 964
6	1,419	57 901	18	1,349	56 940
7	1,394	57 770	19	1,354	56 873
8	1,381	57 726	20	1,357	56 856
9	1,369	57 663	21	1,357	56 858
10	1,361	57 596	22	1,351	56 834
11	1,356	57 559	23	1,347	56 818
12	1,354	57 554	24	1,349	56 814

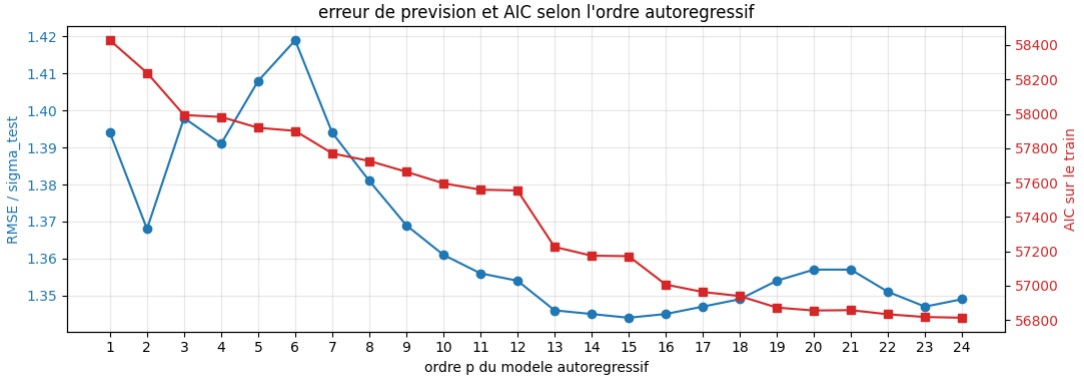


FIGURE 22 – Erreur de prévision à 24 pas et AIC en fonction de l'ordre autorégressif.

La courbe d'erreur n'est pas monotone. Elle descend jusqu'à $p = 2$, remonte jusqu'à $p = 6$ où elle atteint 1,419, puis redescend régulièrement et se stabilise à partir de $p = 13$ autour de 1,345. L'ordre $(2, 0, 0)$ que j'avais retenu se trouve donc être un optimum local : meilleur que ses

voisins immédiats, mais moins bon que les ordres élevés. L'écart reste faible, 0,024 sur l'erreur normalisée. Je n'ai pas d'explication établie pour la bosse autour de $p = 6$.

L'AIC, lui, décroît de façon monotone sur toute la plage, de 58 428 à 56 814. Il ne présente aucun minimum intérieur. C'est la conséquence directe de la faiblesse de la pénalité annoncée en section 3 : entre $p = 1$ et $p = 24$, l'ajustement gagne 1 614 points d'AIC pour 23 paramètres de plus, soit 70 par paramètre contre une pénalité de 2.

5.2.8 Un piège dont j'ai été victime

Un ARIMA estime, en plus de ses coefficients, la moyenne autour de laquelle la série oscille. Statsmodels l'ajoute par défaut quand on ne différencie pas. C'est un paramètre comme les autres, donc il part avec eux quand on transfère le modèle. Sur la série non recentrée, il vaut 11,44 pour Seattle et 24,87 pour Miami : appliquer le modèle de Seattle à Miami revient à prédire 13 degrés trop bas.

Le recentrage par ville fait disparaître cette différence : une fois la moyenne d'apprentissage retirée, l'ARIMA estime une moyenne nulle partout, et transférer zéro ne change rien.

Un second piège vient de l'appariement des paramètres. Un ARIMA$(2, 0, 0)$ et un ARIMA$(2, 1, 1)$ en comptent chacun 4, mais ces 4 paramètres ne désignent pas les mêmes choses. La bibliothèque accepte le transfert sans rien dire et décale tous les rôles d'un cran. Les coefficients ainsi mélangés ne respectent plus la condition de stationnarité, et la prévision, au lieu de revenir vers la moyenne, s'éloigne un peu plus à chaque pas : au bout de 24 pas elle n'a plus aucun sens (divergence). Aucun message n'est émis, alors que 2 ordres de tailles différentes auraient levé une erreur.

5.3 Sur la classification d'activité

5.3.1 Le score global masque les difficultés

Le modèle source atteint 0,958 sur le domaine cible. Décomposé par classe :

activité	rappel
couché	0,998
marche	0,986
descente d'escalier	0,969
debout	0,959
montée d'escalier	0,943
assis	0,888

Décomposé par couple sujet-classe, les 5 cases les plus basses valent 0,65, 0,68, 0,83, 0,84 et 0,84. Un score global de 0,958 contient donc une case à 0,65 : un des sujets rate un tiers de ses montées d'escalier.

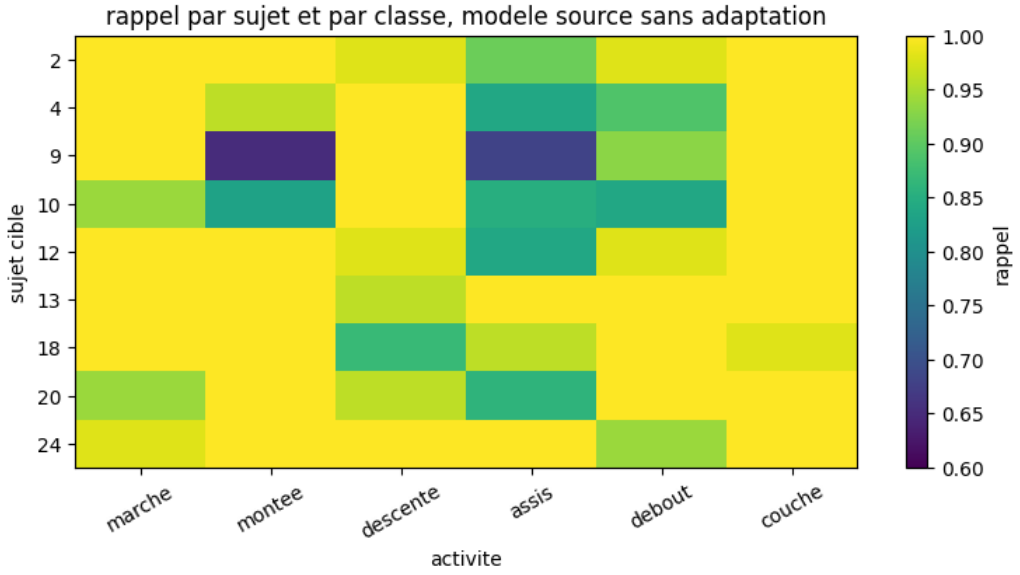


FIGURE 23 – Rappel par sujet et par classe sur le domaine cible. Les cases sombres localisent les difficultés que le score global dissimule.

5.3.2 La frontière mobile-immobile

La matrice de confusion montre un résultat net : les erreurs se répartissent en deux confusions dominantes, entre assis et debout d'une part, entre montée et descente d'escalier d'autre part. Aucune ne confond une activité mobile avec une activité immobile.

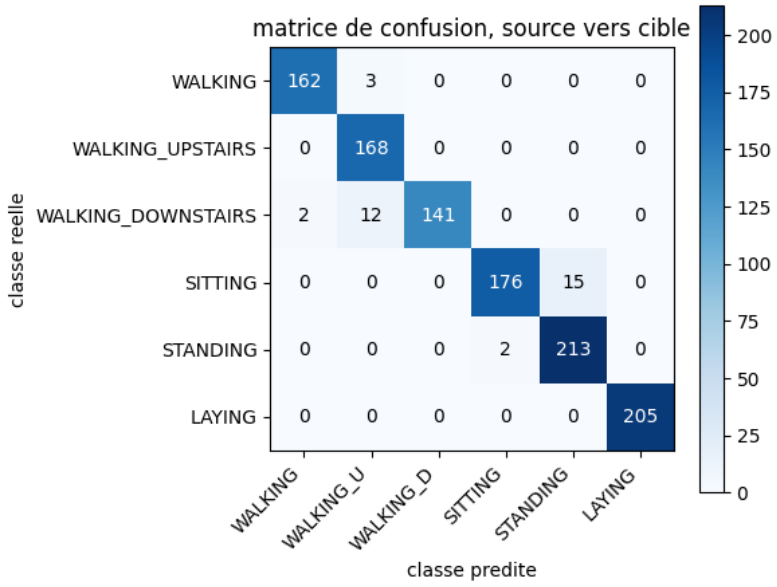


FIGURE 24 – Matrice de confusion sur les 3 sujets cible de test, et non sur les 9, d'où des rappels différents de ceux du tableau précédent.

5.3.3 Deux sujets difficiles, deux causes différentes

Les deux sujets les plus mal classés en validation croisée obtiennent 0,768 et 0,814, contre 0,994 pour le meilleur.

J'ai d'abord testé l'hypothèse d'un défaut de capteur, par le raisonnement physique décrit en section 4 : la norme du vecteur gravité pendant les activités statiques doit valoir un. Elle vaut entre 1,014 et 1,024 pour les 21 sujets, les deux difficiles compris. Le capteur fonctionne donc normalement.

Le premier sujet concentre 59 de ses 75 erreurs sur une seule confusion, sa marche à plat classée comme une montée d'escalier : c'est probablement un problème de démarche. Le second concentre 58 de ses 68 erreurs sur la paire assis-debout : c'est sans doute un problème de posture.

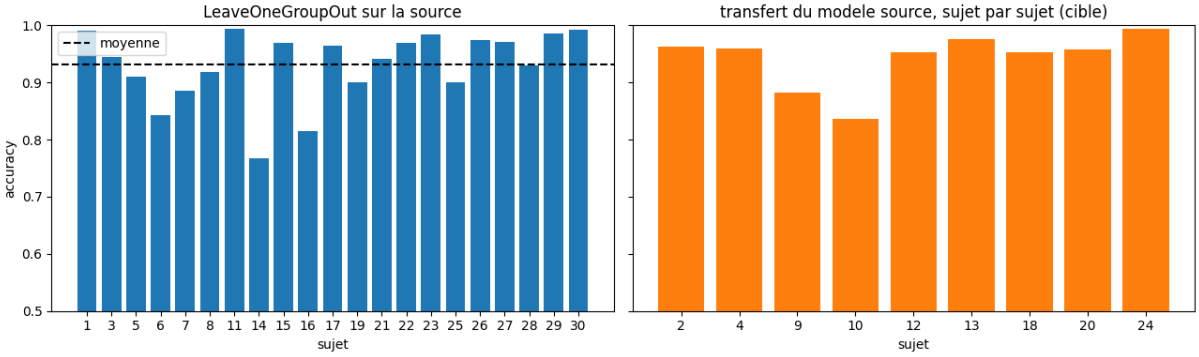


FIGURE 25 – Performance sujet par sujet. À gauche, chaque sujet source évalué par les 20 autres. À droite, chaque sujet cible évalué par le modèle source complet. Les deux panneaux ne mesurent pas la même chose : le premier est une validation croisée, le second un transfert.

5.3.4 Un sujet difficile à prédire est un bon sujet source

Ce résultat est contre-intuitif et je l'ai vérifié soigneusement.

domaine source utilisé	fenêtres	performance cible
les 21 sujets	7 352	0,958
19 sujets, sans les deux plus difficiles	6 663	0,934
les 2 plus difficiles seulement	689	0,745

Retirer les 2 sujets les plus difficiles dégrade le transfert. Comme cela retire aussi des données, j'ai contrôlé l'effet de quantité en retirant deux sujets tirés au hasard, 15 fois. La performance moyenne est alors de 0,952, et aucun tirage ne descend en dessous de 0,938.

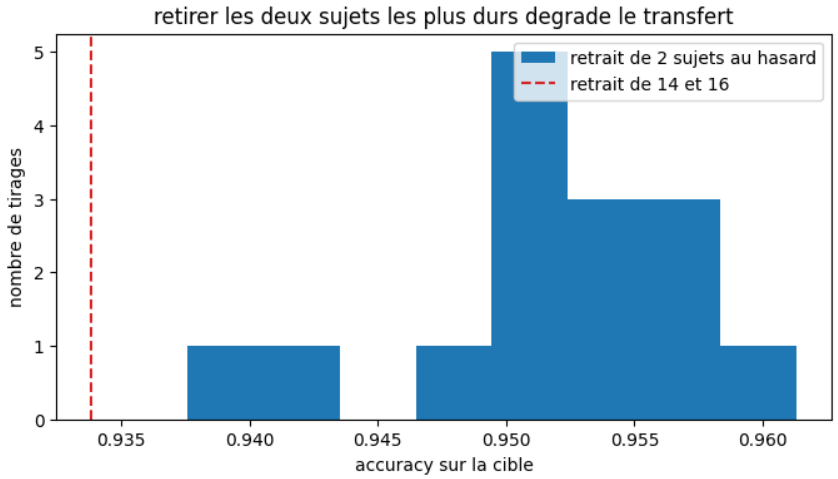


FIGURE 26 – Distribution des performances obtenues en retirant deux sujets au hasard, comparée au retrait ciblé des deux sujets les plus difficiles.

La validation croisée mesure une chose précise : la facilité avec laquelle les autres sujets prédisent celui qu'on met de côté. Elle ne dit rien de sa valeur comme exemple d'entraînement. Un sujet dont la marche est inhabituelle est mal prédit par les autres, mais il élargit ce que le modèle a vu, et le rend meilleur sur les sujets atypiques.

Le domaine cible donne le même enseignement. Le sujet qui y paraît le plus difficile, à 0,751 en validation interne, atteint 0,958 sous le modèle source complet. Sa difficulté venait du protocole, qui ne lui laissait que 8 sujets pour apprendre, et non de lui.

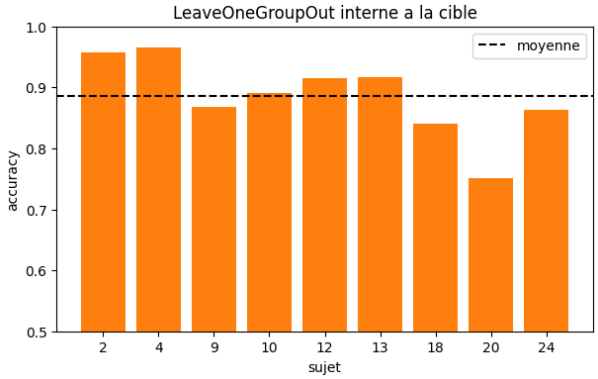


FIGURE 27 – Validation croisée interne au domaine cible, où chaque sujet est évalué par les 8 autres. Le classement obtenu ne survit pas au transfert : le sujet le plus faible ici gagne 0,207 sous le modèle source, et l'un des moins bien classés devient le mieux classé de tous, à 0,995.

La validation croisée sert à voir la variabilité entre sujets, pas à les classer.

5.3.5 Combien de données cible faut-il

Une fois les domaines équilibrés, à 6 sujets source contre 6 sujets cible, j'ai tracé quatre courbes d'apprentissage en fonction du nombre de fenêtres utilisées.

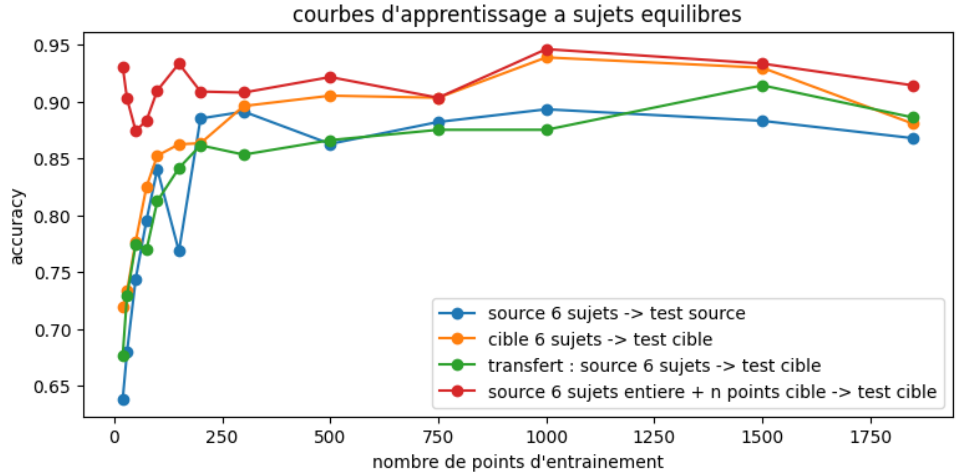


FIGURE 28 – Courbes d'apprentissage à domaines équilibrés, 6 sujets de chaque côté.

Les 3 premières courbes se rejoignent quand on leur donne la même quantité de données : à 1 848 fenêtres, 0,868 pour la source, 0,881 pour la cible et 0,886 pour le transfert. L'avantage que le modèle source semblait avoir dans le découpage officiel venait donc de sa taille, 21 sujets contre 9, et non du domaine.

La quatrième courbe part de 0,931 avec seulement 20 fenêtres cible, parce qu'elle dispose en plus de toute la source. Ajouter des données cible ne lui apporte ensuite presque rien.

J'ai poussé ce cas jusqu'à l'extrême en réduisant la source à 100 fenêtres.

fenêtres cible	cible seule	avec 100 fenêtres source
2	0,324	0,821
5	0,650	0,823
10	0,554	0,832
15	0,642	0,811

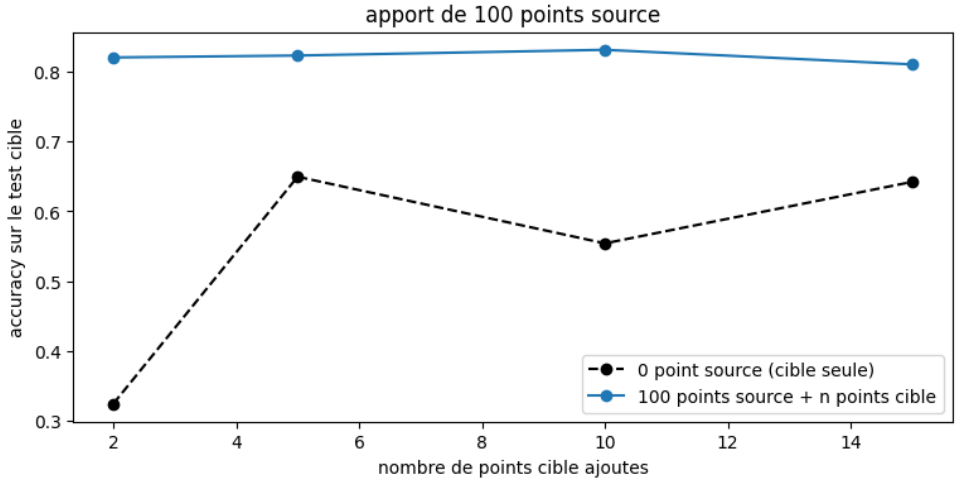


FIGURE 29 – Apport de 100 fenêtres source quand la cible n'en fournit que quelques-unes.

Avec 2 fenêtres cible, 100 fenêtres source font passer le score de 0,324 à 0,821. La colonne de gauche n'est d'ailleurs même pas croissante, 0,650 puis 0,554 : à cette échelle, le tirage compte

sans doute plus que la quantité. La source, elle, stabilise le résultat autour de 0,82 quel que soit le nombre de fenêtres cible.

Ici la source est faible et la cible presque vide, et le transfert apporte 50 points. Dans l'expérience qui suit, la source est forte et le transfert n'apporte rien.

5.3.6 L'expérience du nouveau sujet

Avec un modèle source appris sur les 21 sujets, qui atteint déjà 0,956, l'adaptation sur 3 fenêtres par activité n'aide jamais.

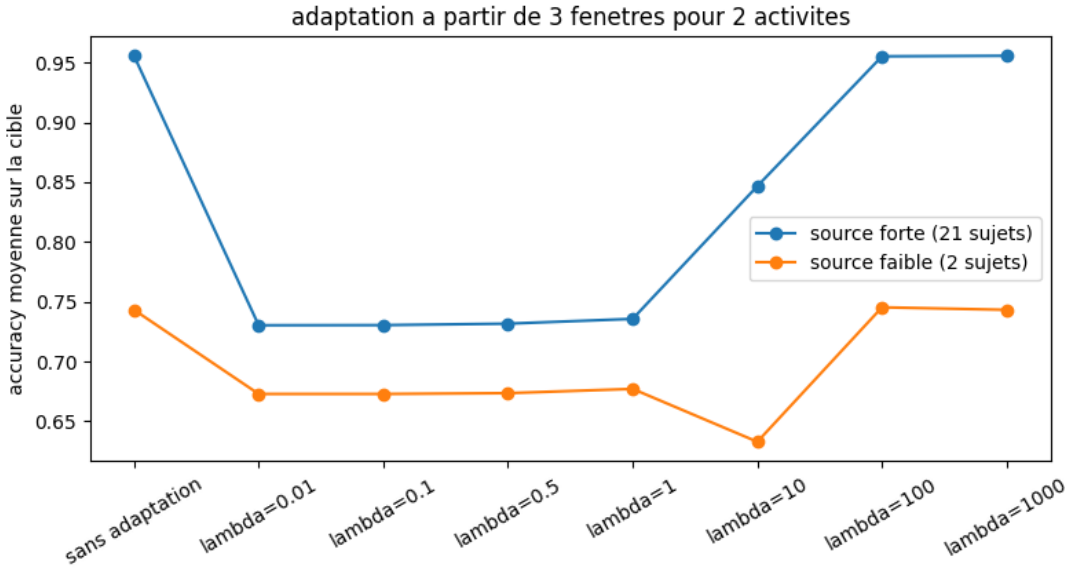


FIGURE 30 – Performance en fonction du rappel vers la source, pour un modèle source fort et un modèle source faible.

Mon hypothèse est que l'adaptation a besoin d'un point de départ moins bon. Avec 7 352 fenêtres source, trois fenêtres cible ne peuvent rien apporter.

5.3.7 Les trois familles d'ADAPT, vues de loin puis de près

J'ai comparé les 3 méthodes sur ce jeu, en faisant varier la source sur 5 tirages de 6 sujets et en évaluant la cible en LeaveOneGroupOut, de sorte que les 9 sujets passent par le test.

méthode	global	marche	montée	descente	assis	debout	couché
sans adaptation	0,899	0,918	0,832	0,858	0,896	0,902	0,975
CORAL	0,564	0,388	0,285	0,361	0,420	0,953	0,877
KMM	0,883	0,956	0,819	0,808	0,813	0,886	0,992
RegularTransferLC	0,938	0,950	0,969	0,948	0,892	0,926	0,948
modèle appris sur la cible	0,883	0,869	0,845	0,764	0,890	0,902	0,994

Vu de loin, le classement est simple. CORAL détruit le modèle, KMM ne change rien, et RegularTransferLC gagne 4 points sur le transfert direct. C'est aussi la seule méthode qui dépasse le modèle appris sur la cible, ce qu'aucune autre expérience de ce stage n'avait montré.

Trois sujets suffisent à voir ce que la moyenne efface. Voici, pour chacun, la classe qui pose problème.

sujet	mesure	sans adapt.	CORAL	KMM	RegTransferLC	cible
18	score global	0,887	0,446	0,874	0,965	0,841
18	descente	0,491	0,033	0,436	**0,862**	0,255
20	score global	0,898	0,641	0,875	0,884	0,751
20	descente	0,662	0,338	0,573	**0,836**	0,289
12	score global	0,879	0,561	0,874	0,959	0,916
12	marche	0,644	0,380	**0,876**	**0,988**	0,980

Le sujet 18 est le cas le plus net. Le transfert direct rate une descente sur deux. Le modèle appris sur la cible fait pire encore, à 0,255. Seule l'adaptation s'en sort, à 0,862. Davantage de données cible n'aurait donc pas suffi.

Le sujet 20 montre ce que le score global cache. Sa descente monte de 0,662 à 0,836, alors que son score global baisse. Un chiffre agrégé aurait conclu que l'adaptation lui nuit.

Le sujet 12 est le seul où deux méthodes réparent. Sa marche passe de 0,644 à 0,876 avec KMM et à 0,988 avec RegularTransferLC. Le modèle appris sur la cible y réussit aussi, à 0,980 : c'est donc bien le transfert qui échouait, et l'adaptation le corrige.

5.3.8 Pourquoi CORAL échoue

CORAL est la seule méthode à dégrader franchement le transfert, en le faisant passer de 0,899 à 0,564.

CORAL cherche à donner aux entrées source la même forme qu'aux entrées cible. Elle calcule pour cela une matrice de covariance, qui décrit la dispersion du nuage de points et ses directions. Ce calcul retire toujours la moyenne au passage, donc la matrice ne décrit que la forme du nuage, jamais sa position.

Dans ADAPT, cette matrice est ensuite appliquée aux données telles quelles, sans leur retirer leur moyenne. La transformation traite donc la position comme si c'était de la forme. Les covariances sont bien alignées, mais les moyennes sont déplacées.

J'ai alors retiré à chaque domaine sa propre moyenne avant d'appliquer la méthode en reprenant le protocole de la section précédente, 5 tirages de 6 sujets source et validation croisée par sujet cible.

variante	global	marche	montée	descente	assis	debout	couché
sans adaptation	0,899	0,918	0,832	0,858	0,896	0,902	0,975
CORAL brut	0,564	0,388	0,285	0,361	0,420	0,953	0,877
CORAL centré	0,897	0,887	0,890	0,855	0,872	0,882	0,984

Le centrage rétablit la méthode, de 0,564 à 0,897, et sur les 9 sujets cible sans exception. Le sujet 4 passe de 0,523 à 0,920, le sujet 13 de 0,653 à 0,957.

D'abord, les bons scores de la version brute étaient trompeurs. Elle obtenait 0,953 en position debout, mieux que toutes les autres méthodes, mais en prédisant cette classe 23,4 % du temps pour 18,1 % de vérité. Une fois centrée, elle retombe à 0,882, c'est-à-dire au niveau du transfert direct. Ce n'était pas une réussite sur cette classe, seulement un déséquilibre des prédictions qui la favorisait.

Ensuite, une fois le défaut corrigé, CORAL n'apporte plus rien : 0,897 contre 0,899 sans adaptation. Tout l'écart observé venait du centrage. La méthode ne dégrade plus, mais elle n'améliore pas non plus, et RegularTransferLC reste très au-dessus avec 0,938.

6 Écrire les décalages pour des variables non indépendantes

Les 3 formes de la section 2.3 sont définies pour des observations tirées indépendamment les unes des autres. Les données de ce stage ne le sont pas : une entrée et sa cible sont découpées dans la même série, et 2 entrées voisines se recouvrent. Cette section écrit donc les décalages dans ce cadre, en distinguant les 2 dépendances en jeu.

6.1 Ce que les articles écrivent

RAINCOAT [11] donne les 2 définitions dont j'ai besoin. Il y a décalage de variables quand

$$p_{\mathcal{S}}(x) \neq p_{\mathcal{T}}(x) \qquad \text{et} \qquad p_{\mathcal{S}}(y \mid x) = p_{\mathcal{T}}(y \mid x),$$

et décalage de classes quand $p_{\mathcal{S}}(y) \neq p_{\mathcal{T}}(y)$. L'article [12] écrit le décalage de covariables de la même façon. Dans les 2 cas, x désigne une série, prise comme une observation unique.

L'article [9] est le seul à traiter la prévision. Il écrit la loi du point suivant sachant tout le passé,

$$P_{\mathcal{S}}(z_{t+1} \mid z_1, \ldots, z_t) = P_{\mathcal{T}}(z_{t+1} \mid z_1, \ldots, z_t),$$

et reproche aux autres approches de regarder les liens entre la sortie, l'entrée et le domaine sans regarder les liens à l'intérieur de l'entrée.

L'article [13] pose le même constat de départ, à savoir que les séries réelles ne sont pas des observations indépendantes, et l'article [14] traite le changement de relation au fil du temps sous le nom de dérive de concept.

Sur l'indépendance entre observations, les articles ne disent pas la même chose. L'article [9] suppose explicitement que les couples (X_i, y_i) sont indépendants et identiquement distribués, alors même qu'il reproche aux autres d'ignorer la structure interne de l'entrée. RAINCOAT [11] et l'article [12] écrivent seulement que les données sont tirées d'une loi, sans se prononcer sur l'indépendance. L'article [13] est le seul à désigner le caractère non indépendant des séries réelles comme le problème à traiter, mais il l'entend comme une dérive au fil du temps à l'intérieur d'un domaine, et non comme une dépendance entre exemples d'apprentissage.

6.2 Deux dépendances à distinguer

Aucune de ces écritures ne tient telle quelle sur mes données, et pour 2 raisons qu'il faut séparer.

La première est la dépendance à l'intérieur d'un couple. En prévision, y_i est un point futur de la série qui a produit X_i, donc la cible n'est pas une quantité extérieure. Changer la loi des fenêtres change du même coup celle des cibles. Cette dépendance disparaît en classification, où l'étiquette est relevée par un opérateur.

La seconde est la dépendance entre couples, et celle-là est présente dans les 2 cas. Deux fenêtres voisines partagent une partie de leurs valeurs, donc X_i et X_j ne sont pas indépendantes.

En classification, les fenêtres sont extraites avec un recouvrement de 50 %, si bien que 2 fenêtres consécutives partagent 64 de leurs 128 mesures. Sur les 10 297 couples de lignes voisines, 9 442 se suivent réellement dans le temps.

En prévision c'est encore plus marqué. La fenêtre avance d'un pas à la fois, donc 2 fenêtres consécutives partagent 23 de leurs 24 valeurs. Et la cible elle-même est réutilisée : $y_i = \eta_{i+47}$ est une coordonnée des 24 fenêtres X_j pour j allant de $i + 24$ à $i + 47$. Les entrées et les cibles ne sont donc pas seulement corrélées, ce sont les mêmes nombres employés à des places différentes.

Autrement dit, l'hypothèse d'indépendance entre observations est fausse dans les 2 expériences, et prendre la fenêtre entière comme observation ne la rétablit pas. Ce que ce changement d'unité règle, c'est uniquement la première dépendance.

La conséquence est la même des 2 côtés : le nombre d'observations n'est pas celui qu'on croit. En classification, avec un recouvrement de moitié et 857 tronçons, le compte effectif est plus proche du nombre de tronçons que des 2 947 fenêtres cible. En prévision, chaque couple (X_i, y_i) occupe 48 points de la série, de η_i à η_{i+47}, donc on ne peut pas extraire plus de 926 couples sans aucune valeur commune sur les 44 460 points disponibles, contre 44 413 fenêtres effectivement construites. Et ces 926 ne sont pas indépendants pour autant, puisque la saisonnalité annuelle relie toute la série.

6.3 En classification

La première dépendance étant absente, les définitions de RAINCOAT [11] s'appliquent directement, avec x la fenêtre de 128 mesures et y l'activité relevée.

La seconde demeure, et rien dans le protocole ne la supprime. Ce qui rend l'évaluation honnête n'est pas l'indépendance mais le découpage par sujet : toutes les fenêtres d'une personne partent du même côté, donc l'apprentissage et le test ne partagent aucune mesure brute. Ce qui reste faux, c'est le nombre d'observations, comme indiqué en section 6.2.

6.4 En prévision

En prévision la première dépendance de la section 6.2 est présente, donc le raccourci précédent ne s'applique plus.

Pour écrire les décalages il faut donc partir de ce qui engendre la série. Je décris un domaine par

$$z_t = g(t) + f(z_{t-1}, \dots, z_{t-p}) + \varepsilon_t, \qquad \varepsilon_t \text{ indépendants, d'écart-type } \sigma, \tag{1}$$

où g est la partie déterministe, tendance et saisonnalité, f la mémoire, et σ l'aléa. La partie g n'apparaît dans aucun des 5 articles ; je l'ajoute parce que mes séries en ont une et que 4 de mes 6 décalages agissent dessus.

Le fenêtrage de la section 3.10 donne $X_i = (z_i, \dots, z_{i+w-1})$ et $y_i = z_{i+w+h-1}$. À l'horizon $h = 1$, les arguments de f sont exactement les p dernières coordonnées de X_i, que je note $x_{(p)}$. La cible s'écrit alors comme l'équation (1) lue sur la fenêtre :

$$y = g(t) + f\big(x_{(p)}\big) + \varepsilon_t, \qquad t = i + w. \tag{2}$$

C'est cette écriture qui me permet de mettre les décalages en équation, un domaine étant décrit par le triplet (g, f, σ) et chaque décalage se lisant sur l'un des 3 termes.

6.5 Les 6 décalages écrits en équation

Les 4 premiers décalages ne touchent que g, en laissant f et σ inchangés. En notant

$$\delta(t) = g_{\mathcal{T}}(t) - g_{\mathcal{S}}(t),$$

l'équation (2) donne pour tous les 4 la même forme :

$$P_{\mathcal{T}}\big(y \mid X = x\big) = P_{\mathcal{S}}\big(y - \delta(t) \mid X = x\big).$$

Ils ne se distinguent que par la forme de δ.

décalage	ce qui change	écart $\delta(t)$
niveau	une constante ajoutée	Δ
tendance	la pente	$(a_{\mathcal{T}} - a_{\mathcal{S}})\, t$
échelle	l'amplitude du cycle	$(A_{\mathcal{T}} - A_{\mathcal{S}}) \sin(2\pi t/\tau)$
fréquence	la période du cycle	$A\big[\sin(2\pi t/\tau_{\mathcal{T}}) - \sin(2\pi t/\tau_{\mathcal{S}})\big]$

Les 2 derniers touchent les autres termes :

$$\text{bruit :} \quad y = g(t) + f\big(x_{(p)}\big) + \varepsilon_t, \qquad \varepsilon_t \text{ d'écart-type } \sigma_{\mathcal{T}},$$

avec la même espérance conditionnelle et une variance différente, et

$$\text{mémoire :} \quad y = g(t) + f_{\mathcal{T}}\big(x_{(p)}\big) + \varepsilon_t,$$

avec une espérance conditionnelle différente. La mémoire porte en réalité sur le bruit. Reportée sur la série, elle se répartit entre f et le terme déterministe.

$$z_t = g(t) + u_t, \qquad u_t = \varphi\, u_{t-1} + \varepsilon_t,$$

donne, en éliminant u,

$$z_t = \underbrace{g(t) - \varphi\, g(t-1)}_{\text{terme déterministe}} + \underbrace{\varphi\, z_{t-1}}_{f} + \varepsilon_t.$$

Trois conséquences se lisent sur ces équations.

D'abord, les 6 décalages sont des décalages de concept, puisque la loi conditionnelle change dans les 6 cas. La catégorie de décalage de covariables ne sert donc à rien ici : entrée et cible venant de la même série, on ne peut pas déplacer l'une sans l'autre.

Ensuite, le décalage de classes n'est pas un axe disponible en prévision. La loi de la sortie s'obtient par

$$P(y) = \int P(y \mid X = x)\, P(X = x)\, dx,$$

donc elle est entièrement déterminée par les 2 autres et ne peut pas être modifiée séparément. Elle n'a de sens qu'en classification.

Enfin, le décalage de niveau est un cas particulier, car il déplace aussi les entrées :

$$P_{\mathcal{T}}(X \in B) = P_{\mathcal{S}}(X \in B - \Delta\mathbf{1}), \qquad P_{\mathcal{T}}(y \mid X = x) = P_{\mathcal{S}}(y - \Delta \mid X = x - \Delta\mathbf{1}).$$

Il change donc les 2 lois à la fois.

Puisque les 6 décalages sont des décalages de concept, la classification habituelle ne les sépare plus. Les équations, elles, les séparent, en distinguant ce qui change dans la loi conditionnelle.

Quand seul σ change, l'espérance conditionnelle est la même, donc le meilleur prédicteur possible est le même. Quand f change, l'espérance change, donc le meilleur prédicteur change aussi. Quand g change, l'espérance est décalée de $\delta(t)$.

6.6 Où se placent les 2 décalages réels

Le changement de ville agit sur g, avec δ constant. La série désaisonnalisée de la ville cible est en moyenne 11,00 degrés au-dessus de celle des villes source. Le recentrage le confirme : en retirant à chaque ville sa propre moyenne, le transfert passe de 1,639 à 1,348, contre 1,328 pour un modèle appris sur place. Une translation récupère presque tout l'écart, ce qui ne serait pas possible si f ou σ avaient changé.

Le changement de sujet est un décalage de variables au sens de RAINCOAT [11]. Ici la fenêtre est l'observation, donc les définitions s'appliquent telles quelles. Il n'y a pas de décalage de classes : sur les 9 sujets cible, l'écart maximal à la répartition uniforme de 16,7 % est de 4 points. Il n'y a pas non plus de changement de relation, et l'argument vient des courbes d'apprentissage à domaines équilibrés, où le modèle transféré atteint 0,886 et le modèle appris sur la cible 0,881. Il reste la loi des entrées, qui change bien d'une personne à l'autre, ce que confirme le comportement de CORAL : la moyenne des entrées se déplace assez pour que la méthode s'y perde, et un recentrage par domaine suffit à la corriger.

7 Conclusion

7.1 Ce que j'ai fait

J'ai suivi les 5 étapes prévues. Prise en main d'ADAPT, construction d'un générateur de séries à décalage contrôlé, mise en place d'un pipeline de référence, comparaison de 9 méthodes sur données synthétiques, puis étude approfondie de 2 jeux réels, un de prévision et un de classification.

Les décalages qui déplacent la série, niveau, échelle et tendance, ne coûtent rien : ce sont des transformations affines que le modèle absorbe, et sur les données réelles un simple recentrage suffit à corriger un écart de 11 degrés. Le bruit et la fréquence changent en revanche ce qu'il y a à prédire. Le R^2 tombe de 0,951 sur la source à 0,800 et 0,621 sur la cible, ce qui multiplie par 4 et par 8 la part de variance que le modèle n'explique pas. C'est sur ces décalages-là que l'adaptation a une chance de servir.

7.2 Ce qui reste incomplet

L'objectif était double : caractériser les décalages, et déterminer quelle méthode convient à quel décalage. Les 9 méthodes ont été comparées sur les 6 décalages et les 3 intensités. Ce rapport ne présente que la comparaison portant sur la fréquence, parce que c'est le seul décalage synthétique où la dégradation laisse quelque chose à récupérer : sur le niveau, l'échelle et la tendance, le modèle de référence ne perd rien, et une méthode d'adaptation n'aurait rien à corriger. Sur les jeux réels, je me suis en revanche limité à 3 méthodes, une par famille, ce qui est la principale limite du banc d'essai.

2 pistes sont restées ouvertes. Le recentrage par fenêtre plutôt que par domaine, qui traiterait aussi la dérive à l'intérieur d'un domaine. Et l'erreur d'extrapolation de la saisonnalité, qu'aucun prolongement construit sur le seul apprentissage ne peut supprimer, puisqu'elle mesure l'imprévisibilité de l'année de test.

7.3 Perspectives

Traiter le recentrage comme une méthode d'adaptation à part entière et le comparer aux méthodes d'ADAPT dans les mêmes conditions, en incluant sa variante par fenêtre.

Faire varier l'horizon de prévision et la taille de la fenêtre glissante utilisée pour la régression linéaire.

7.4 Ce que je laisse

Un ensemble de notebooks et 2 modules réutilisables, couvrant la génération de séries à décalage contrôlé, le pipeline de référence et l'évaluation des méthodes d'ADAPT sur 2 tâches.

Quatre biais d'évaluation chiffrés : le choix du dénominateur, la fuite d'information de la décomposition, la lecture du coefficient de détermination entre domaines, et l'écart entre score global et score par classe. Ils concernent toute évaluation d'adaptation sur séries temporelles, quelles que soient les méthodes comparées. C'est l'apport qui me paraît le plus durable.

Enfin un résultat à vérifier plus largement : un recentrage d'une ligne bat 2 des 3 méthodes testées en prévision. Il pourrait servir de référence de base dans un banc d'essai futur.

Références

[1] A. de Mathelin, M. Atiq, G. Richard, A. de la Concha, M. Yachouti, F. Deheeger, M. Mougeot, N. Vayatis. *ADAPT : Awesome Domain Adaptation Python Toolbox.* arXiv :2107.03049, 2023. https://adapt-python.github.io/adapt/

[2] L. Oudre. *Machine Learning for Time Series*, Master MVA, ENS Paris-Saclay, 2025–2026. Notamment la séance 3, Models and Representation Learning.

[3] D. Anguita, A. Ghio, L. Oneto, X. Parra, J. L. Reyes-Ortiz. *A Public Domain Dataset for Human Activity Recognition Using Smartphones*. ESANN, Bruges, 2013. https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones

[4] R. B. Cleveland, W. S. Cleveland, J. E. McRae, I. Terpenning. *STL : A Seasonal-Trend Decomposition Procedure Based on Loess*. Journal of Official Statistics, 6(1), 1990, p. 3–73.

[5] S. Seabold, J. Perktold. *statsmodels : Econometric and statistical modeling with Python*. Proceedings of the 9th Python in Science Conference, 2010.

[6] F. Pedregosa et al. *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, 12, 2011, p. 2825–2830.

[7] Y. Shi, X. Ying, J. Yang. *Deep Unsupervised Domain Adaptation with Time Series Sensor Data : A Survey*. Sensors, 22(15), 2022. https://doi.org/10.3390/s22155507

[8] M. Ragab, E. Eldele, W. L. Tan, C.-S. Foo, Z. Chen, M. Wu, C.-K. Kwoh, X. Li. *AdaTime : A Benchmarking Suite for Domain Adaptation on Time Series Data*. ACM Transactions on Knowledge Discovery from Data, 2023. https://doi.org/10.1145/3587937

[9] Z. Li, R. Cai, T. Z. J. Fu, Z. Hao, K. Zhang. *Transferable Time-Series Forecasting under Causal Conditional Shift*. IEEE TPAMI, 2023. https://arxiv.org/abs/2111.03422

[10] B. Lambert, C. L. Lei, M. Robinson, M. Clerx, R. Creswell, S. Ghosh, S. Tavener, D. J. Gavaghan. *Autocorrelated measurement processes and inference for ordinary differential equation models of biological systems*. Journal of the Royal Society Interface, 20(199) :20220725, 2023. https://doi.org/10.1098/rsif.2022.0725

[11] H. He, O. Queen, T. Koker, C. Cuevas, T. Tsiligkaridis, M. Zitnik. *Domain Adaptation for Time Series Under Feature and Label Shifts*. ICML, 2023. https://arxiv.org/abs/2302.03133

[12] F. Ott, D. Rügamer, L. Heublein, B. Bischl, C. Mutschler. *Domain Adaptation for Time-Series Classification to Mitigate Covariate Shift*. ACM International Conference on Multimedia, 2022. https://arxiv.org/abs/2204.03342

[13] H. He, Q. Zhang, K. Yi, K. Shi, Z. Niu, L. Cao. *Distributional Drift Adaptation with Temporal Conditional Variational Autoencoder for Multivariate Time Series Forecasting*. 2023. https://arxiv.org/abs/2209.00654

[14] G. H. F. M. Oliveira, R. C. Cavalcante, G. G. Cabral, L. L. Minku, A. L. I. Oliveira. *Time Series Forecasting in the Presence of Concept Drift : A PSO-based Approach*. IEEE ICTAI, 2017.

[15] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, J. Choo. *Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift*. ICLR, 2022. https://openreview.net/forum?id=cGDAkQo1C0p

Glossaire

Adaptation de domaine Ensemble de méthodes cherchant à transférer vers un domaine cible ce qui a été appris sur un domaine source de distribution différente. Page 1.

Décalage de concept Situation où la relation entre entrée et sortie change d'un domaine à l'autre. Page 7.

Décalage de covariables Situation où la loi des entrées change mais la relation entrée-sortie reste la même. Page 7.

Domaine cible Domaine sur lequel on veut prédire, où les annotations sont rares ou absentes. Page 7.

Domaine source Domaine sur lequel on dispose de données annotées. Page 7.

Fenêtre glissante Découpage d'une série en segments consécutifs de longueur fixe, pour la ramener à un problème supervisé. Page 16.

Horizon Nombre de pas séparant la fin de la fenêtre d'entrée de la valeur à prédire. Page 9.

Innovation Écart entre l'observation et sa prédiction à un pas, servant d'estimation du choc inobservé. Page 10.

LeaveOneGroupOut Validation croisée mettant un seul groupe de côté à chaque itération. Page 18.

Opérateur retard Opérateur B tel que $Bu_t = u_{t-1}$, permettant d'écrire les modèles sous forme polynomiale. Page 10.

Prévision directe Approche entraînant un modèle à prédire directement à l'horizon voulu. Page 17.

Prévision itérée Approche modélisant un pas et l'appliquant plusieurs fois. Page 23.

Résidu de décomposition Ce qui reste d'une série une fois retirées sa tendance et sa saisonnalité. N'est pas un bruit blanc. Page 9.

Stationnarité Propriété d'une série dont les deux premiers moments ne dépendent pas du temps. Page 13.

STL, MSTL Décompositions saisonnières par lissages locaux successifs, la seconde traitant plusieurs périodes. Page 2.

A Annexe : organisation du code

Le travail est réparti en 23 notebooks et 2 modules partagés, versionnés sur GitHub pendant toute la durée du stage.

Les 2 modules. Ils rassemblent tout ce qui sert à plus d'un notebook, pour qu'une correction se propage partout au lieu d'être recopiée.

`series_utils.py`, produit les séries synthétiques et mesure leurs paramètres : `generate_series` pour une série à tendance, saisonnalité et bruit indépendant, `generate_series_ar` pour la variante dont le bruit a une mémoire, `estimate_period` qui estime la période par le premier pic de l'autocorrélation, et `rmse`.

`adapt_utils.py`, contient le reste : le passage au problème supervisé avec `sliding_window` et `build_train_test`, le découpage par sujet avec `split_har`, l'entraînement et l'évaluation avec `predict` et `compute_metrics`, la comparaison des méthodes d'ADAPT avec `comparer_adapt` et `run_all_methods`, et les courbes d'apprentissage avec `courbe_apprentissage` et `courbe_melange`.

Trois extraits. Le premier transforme une série en problème supervisé.

```
def sliding_window(series, window_size, horizon=1):
    X, y = [], []
    for i in range(len(series) - window_size - horizon + 1):
        # entree : les w valeurs consecutives a partir de i
        X.append(series[i:i + window_size])
        # cible : la valeur situee horizon pas apres la fin de la fenetre
        y.append(series[i + window_size + horizon - 1])
    return np.array(X), np.array(y)
```

Le deuxième est le recentrage, qui tient en deux lignes au milieu de la construction de la série désaisonnalisée. La moyenne est calculée sur la seule période d'apprentissage, ce qui évite d'utiliser une information du test.

```
    z = temp - np.concatenate([sais_app, prolonge])
    # moyenne calculee sur le train uniquement, puis retiree partout
    moyenne_ville[ville] = float(z[:n_app].mean())
    return z - moyenne_ville[ville]
```

Le troisième découpe le domaine par sujet et jamais par ligne. Les masques portent sur le vecteur des sujets, si bien qu'aucune personne ne peut se retrouver des deux côtés.

```
def split_har(Xs, ys, subj_s, Xt, yt, subj_t,
              n_test_source=4, n_train_cible=6):
    src, tgt = sorted(set(subj_s)), sorted(set(subj_t))
    # on selectionne des sujets, pas des lignes
    m_src = np.isin(subj_s, src[-n_test_source:])
    m_tgt_tr = np.isin(subj_t, tgt[:n_train_cible])
    m_tgt_te = np.isin(subj_t, tgt[n_train_cible:])
    return (Xs[~m_src], ys[~m_src], Xs[m_src], ys[m_src],
            Xt[m_tgt_tr], yt[m_tgt_tr], Xt[m_tgt_te], yt[m_tgt_te])
```

B Annexe : les jeux de données

— Historical Hourly Weather Data, températures horaires de 36 villes, pas horaire, d'octobre 2012 à novembre 2017 : kaggle.com, historical hourly weather data
— Human Activity Recognition Using Smartphones, 30 sujets, 6 activités, accéléromètre et gyroscope à 50 Hz : archive.ics.uci.edu, jeu 240

C Annexe : supports de cours utilisés

Deux cours de Laurent Oudre [2] m'ont servi de référence pour les séries temporelles.
— *Machine Learning for Time Series*, séance 3, Models and Representation Learning, Master MVA, ENS Paris-Saclay, 2025–2026. Modèles autorégressifs, moyenne mobile et ARMA, estimation de la période par autocorrélation.
— *Intelligence artificielle et apprentissage automatique pour la modélisation de séries temporelles et de signaux*, séance 3, Modèles pour les séries temporelles, diplôme ARIA, ENS Paris-Saclay, 2025–2026.

D Annexe : pages de référence en ligne

Les pages suivantes (Wikipédia) couvrent les notions employées dans ce rapport.

notion	page
critère d'information d'Akaike	Critère d'information d'Akaike
modèle autorégressif	Processus autorégressif
moyenne mobile	Moyenne mobile
modèle ARIMA	ARIMA
autocorrélation	Autocorrélation
stationnarité	Stationnarité
bruit blanc	Bruit blanc
adaptation de domaine	Adaptation de domaine
transfert d'apprentissage	Transfert d'apprentissage

E Annexe : les décalages synthétiques

Le domaine source est le même pour les 6 décalages. Il est produit par

$$x_t = a\,t + A\sin\Big(\frac{2\pi t}{T}\Big) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2),$$

avec une pente $a = 0{,}01$, une amplitude saisonnière $A = 1{,}0$, une période $T = 50$ et un écart-type de bruit $\sigma = 0{,}15$, sur 800 points découpés en fenêtres de 20.

Le domaine cible reprend la même formule, avec un seul paramètre modifié. L'intensité i règle l'ampleur de cette modification, et prend les valeurs 0,25, 0,50 et 1,00.

décalage	paramètre modifié	$i = 0{,}25$	$i = 0{,}50$	$i = 1{,}00$
niveau	constante ajoutée	+0,25	+0,50	+1,00
échelle	amplitude $A(1+i)$	1,25	1,50	2,00
tendance	pente $a(1+i)$	0,0125	0,0150	0,0200
fréquence	période $T/(1+i)$	40,0	33,3	25,0
bruit	écart-type $\sigma(1+i)$	0,1875	0,2250	0,3000

Le décalage combiné applique les 5 modifications en même temps, à la même intensité (l'intensité n'a pas d'unité commune).

F Annexe DD&RS : développement durable et responsabilité sociétale

F.1 Périmètre et sources

Cette annexe porte sur l'ENSIIE, établissement où le stage s'est déroulé, et plus précisément sur le C19, où le travail a été mené. Elle s'appuie sur les documents publics de l'école : la page consacrée à sa démarche DD&RS, son rapport développement durable 2022–2023, l'annonce du renouvellement de son label en 2025 et ses indicateurs sociaux. Ces documents sont complétés par des publications de l'ADEME, de l'Arcep, de l'Agence internationale de l'énergie et des ministères compétents. Les liens sont donnés en fin d'annexe.

F.2 Pressions exercées sur l'environnement

Le secteur numérique

Une école d'ingénieurs en informatique exerce les pressions communes à tout établissement d'enseignement supérieur, chauffage, électricité, déplacements, restauration et déchets, auxquelles

s'ajoute une dépendance forte aux équipements électroniques et aux centres de données. L'essentiel de l'impact ne vient pas de l'électricité consommée pendant un calcul, mais de la fabrication du matériel : extraction de métaux, consommation d'eau et d'énergie, puis déchets électroniques en fin de vie. Allonger la durée de vie des machines pèse donc au moins autant que d'optimiser leur usage.

L'étude conjointe de l'ADEME et de l'Arcep chiffre ces pressions. En 2022, le numérique représentait en France 29,5 millions de tonnes équivalent CO_2, soit 4,4 % de l'empreinte carbone nationale, et 51,5 TWh, soit 11 % de la consommation électrique. Elle attribue environ 50 % de cette empreinte aux terminaux, 46 % aux centres de données et 4 % aux réseaux, et estime à 117 millions de tonnes les ressources mobilisées chaque année pour produire les équipements. La part des terminaux confirme que la durée de vie du matériel compte au moins autant que la sobriété d'usage.

L'intelligence artificielle accentue la pression sur les centres de données. À l'échelle mondiale, ceux-ci ont consommé environ 415 TWh en 2024, soit 1,5 % de l'électricité mondiale, et le scénario central de l'Agence internationale de l'énergie les porte à environ 945 TWh en 2030. S'y ajoutent des enjeux locaux : puissance disponible sur le réseau, eau de refroidissement et emprise foncière.

L'ENSIIE et le C19

Sur le site, les postes de travail, les salles d'enseignement, les serveurs et les déplacements constituent les principales sources de pression. Le C19 en concentre une partie : l'école y présente un centre de données, un parc informatique, des équipements de réalité virtuelle et augmentée, deux salles immersives et des espaces de travail pour des projets et de jeunes entreprises. Ces moyens sont cohérents avec une activité de formation et d'innovation, mais leur fabrication et leur renouvellement entrent dans l'empreinte du site.

La recherche en apprentissage automatique

Le domaine dans lequel s'inscrit ce stage a une particularité : la comparaison de méthodes multiplie les exécutions. Chaque méthode, chaque jeu de données et chaque intensité de décalage demande un ajustement, et l'on répète l'ensemble à chaque correction du protocole. Le coût total ne tient pas au modèle final mais au nombre d'essais qui ont conduit à lui.

La mission elle-même

Mon travail est resté léger. Tout a tourné sur un ordinateur portable, sur des jeux de données modestes : 44 460 points horaires par ville pour la météo, 10 299 fenêtres pour la classification d'activité. Les modèles sont des régressions linéaires, des ARIMA d'ordre faible et une régression logistique.

Une expérience fait exception, et elle est instructive. Le balayage des ordres autorégressifs demande un ajustement et une prévision pour chaque ordre et chaque origine de test, soit plus de 200 000 prévisions. J'ai réduit ce coût en n'évaluant qu'une origine sur 8, puis j'ai vérifié que cette réduction ne changeait pas les conclusions, l'écart mesuré étant de 0,3 %. Le calcul est ainsi passé de plusieurs heures (sans doute) à 30 minutes. C'est un exemple concret de sobriété obtenue sans perte d'information, mais il faut noter que la vérification a elle-même demandé du calcul.

F.3 Cadre réglementaire

Deux textes encadrent ces pressions, avec des niveaux d'obligation différents.

Le dispositif Éco Énergie Tertiaire, issu de la loi Élan, s'applique aux bâtiments accueillant au moins 1 000 m^2 d'activités tertiaires, ce qui inclut l'enseignement et les salles informatiques.

Il impose une réduction de la consommation d'énergie finale d'au moins 40 % en 2030, 50 % en 2040 et 60 % en 2050 par rapport à 2010, avec un suivi annuel sur la plateforme OPERAT.

Pour le numérique, le Référentiel général d'écoconception des services numériques propose 78 critères portant sur l'utilité du service, l'architecture, l'hébergement, la limitation des ressources et la transparence environnementale.

F.4 Politique de l'établissement

L'ENSIIE s'est dotée d'une charte DD&RS en 2019. Sa démarche est portée par la direction, une personne chargée de mission et un comité de pilotage, et couvre les 5 axes du référentiel national : stratégie et gouvernance, formation, recherche, environnement et politique sociale. L'école a obtenu le renouvellement de son label DD&RS en 2025 pour 4 ans, l'audit relevant notamment la gouvernance, l'implication des élèves, la recherche liée à la transition, la végétalisation et la politique sociale.

Parmi les réalisations publiées figurent un bilan carbone portant sur 2018, un plan de mobilité inter-établissements, le tri des déchets par filières, un atelier de réparation informatique et la revégétalisation du square.

Du côté de la formation, le rapport 2022–2023 indique que tous les élèves ont suivi un enseignement DD&RS depuis 2019 et que 170 élèves ont participé à la rentrée climat. Il cite des Fresques du climat, des ateliers 2tonnes et un cours sur les impacts environnementaux du numérique. Onze enseignants-chercheurs avaient déclaré des thématiques liées au DD&RS, et l'école participe à Labos 1point5 et au GDS EcoInfo.

F.5 Lien entre la mission et la sobriété

L'adaptation de domaine est, par nature, une démarche de réemploi. Plutôt que de réapprendre un modèle pour chaque nouveau contexte, on cherche à réutiliser celui dont on dispose déjà. Sur la tâche de prévision, un recentrage par ville, qui tient en une ligne et ne demande aucune donnée annotée de la ville cible, amène le transfert à 1,5 % du modèle appris sur place. Sur la classification, une méthode fondée sur les paramètres atteint le niveau d'un modèle cible avec très peu d'étiquettes nouvelles.

Ces résultats vont dans le sens d'une réduction du coût d'apprentissage. Ils appellent toutefois deux réserves. La première est que la méthode la plus sobre s'est aussi révélée la plus efficace, ce qui n'était pas acquis d'avance et ne se généralise pas. La seconde est que je n'ai mesuré aucune consommation d'énergie : je décris un potentiel de réduction du calcul, pas une économie constatée.

F.6 Responsabilité sociétale

Politique sociale de l'établissement

La politique sociale de l'ENSIIE couvre l'égalité, la diversité, l'accessibilité et la prévention des violences. Les indicateurs publiés donnent 20 % de femmes parmi les élèves en 2024–2025, 18 % parmi les diplômés de la même promotion, et un salaire moyen à 24 mois de 44 377 euros pour les hommes contre 42 373 euros pour les femmes sur la promotion 2023, soit un écart d'environ 4,5 %.

Ces indicateurs rendent le suivi possible, et c'est déjà beaucoup, mais ils confirment que la féminisation reste un objectif à poursuivre.

F.7 Analyse critique

Le principal reproche que je me fais est de n'avoir mesuré aucune consommation. Enregistrer le temps de calcul de chaque expérience aurait coûté peu et aurait permis de comparer les

méthodes sur ce critère comme sur la précision. Une méthode d'adaptation dont le coût dépasse celui d'un réapprentissage complet a peu d'intérêt pratique, et je ne suis pas en mesure de le vérifier pour celles que j'ai testées.

F.8 Sources

document	lien
ENSIIE, démarche développement durable	ensiie.fr/lecole/developpement-durable
ENSIIE, rapport développement durable 2022–2023	rapport_ddrs_2023_ensiie.pdf
ENSIIE, renouvellement du label DD&RS 2025	ensiie.fr, actualité label DD&RS 2025
ENSIIE, indicateurs DD&RS	ensiie.fr/indicateurs-ddrs
ENSIIE, entrepreneuriat et C19	ensiie.fr/entreprise/entrepreneuriat
ADEME et Arcep, *Numérique : quel impact environnemental en 2022 ?*	infos.ademe.fr
Agence internationale de l'énergie, *Energy and AI*	iea.org/reports/energy-and-ai
Arcep, IA générative et défis environnementaux	arcep.fr, rapport IA et environnement
Arcep et Arcom, référentiel d'écoconception	arcep.fr, RGESN version 2024
Ministère de la Transition écologique, Éco Énergie Tertiaire	ecologie.gouv.fr, Éco Énergie Tertiaire

G Annexe : déclaration d'utilisation de l'intelligence artificielle

J'ai eu recours à l'intelligence artificielle (Gemini) pendant ce stage pour la réalisation des illustrations LaTeX et pour de la correction de codes python (debogage, optimisation car execution trop longue...).

H Annexe : quatre démonstrations

H.1 L'espérance conditionnelle minimise l'erreur quadratique

On veut prédire η_{t+h} à partir de ce qui est connu à l'instant t, noté $\mathcal{F}_t$. Le prédicteur est donc une fonction g de cette information, et on cherche celle qui minimise l'erreur quadratique moyenne.

Posons $m = \mathbb{E}\big[\eta_{t+h} \mid \mathcal{F}_t\big]$. Les quantités m et g ne dépendent que de $\mathcal{F}_t$: elles se comportent donc comme des constantes une fois cette information donnée.

On écrit l'erreur en faisant apparaître m :

$$\mathbb{E}\big[(\eta_{t+h} - g)^2 \mid \mathcal{F}_t\big] = \mathbb{E}\big[\big((\eta_{t+h} - m) + (m - g)\big)^2 \mid \mathcal{F}_t\big].$$

En développant le carré :

$$= \underbrace{\mathbb{E}\big[(\eta_{t+h} - m)^2 \mid \mathcal{F}_t\big]}_{\text{ne dépend pas de } g} + 2\,(m - g)\,\mathbb{E}\big[\eta_{t+h} - m \mid \mathcal{F}_t\big] \;+\; (m - g)^2.$$

Le terme du milieu est nul, car par définition de m :

$$\mathbb{E}\big[\eta_{t+h} - m \mid \mathcal{F}_t\big] = m - m = 0.$$

Il reste

$$\mathbb{E}\big[(\eta_{t+h} - g)^2 \mid \mathcal{F}_t\big] = \mathrm{Var}\big(\eta_{t+h} \mid \mathcal{F}_t\big) + (m - g)^2.$$

Le premier terme ne dépend pas du prédicteur choisi. Le second est un carré, donc positif ou nul, et il s'annule exactement lorsque $g = m$. Le minimum est donc atteint pour

$$\boxed{g = \mathbb{E}\big[\eta_{t+h} \mid \mathcal{F}_t\big]}$$

En prenant l'espérance des deux membres, le même résultat vaut pour l'erreur quadratique moyenne non conditionnelle.

Conséquence pratique. Comme les aléas futurs sont centrés et indépendants du passé, calculer cette espérance revient à remplacer chaque ε futur par zéro. C'est ce qui est fait en section 3.

H.2 Variance de l'erreur de prévision d'un AR(1)

Déroulement de la récursion

On part de $\eta_{t+1} = \varphi\,\eta_t + \varepsilon_{t+1}$ et on montre par récurrence que

$$\eta_{t+h} = \varphi^{\,h}\,\eta_t \;+\; \sum_{j=0}^{h-1} \varphi^{\,j}\,\varepsilon_{t+h-j}.$$

Initialisation. Pour $h = 1$, la somme se réduit au terme $j = 0$, OK.

Hérédité. Supposons la formule vraie au rang h

$$\begin{aligned}
\eta_{t+h+1} &= \varphi\,\eta_{t+h} + \varepsilon_{t+h+1} \\
&= \varphi\left(\varphi^{\,h}\eta_t + \sum_{j=0}^{h-1}\varphi^{\,j}\varepsilon_{t+h-j}\right) + \varepsilon_{t+h+1} \\
&= \varphi^{\,h+1}\eta_t + \sum_{j=0}^{h-1}\varphi^{\,j+1}\varepsilon_{t+h-j} + \varepsilon_{t+h+1}.
\end{aligned}$$

En posant $i = j + 1$, la somme devient $\sum_{i=1}^{h} \varphi^{\,i}\varepsilon_{t+h+1-i}$. Le terme ε_{t+h+1} correspond à $i = 0$, ce qui permet de l'intégrer à la somme :

$$\eta_{t+h+1} = \varphi^{\,h+1}\eta_t + \sum_{i=0}^{h}\varphi^{\,i}\varepsilon_{t+h+1-i}.$$

Variance des chocs à venir

Le premier terme, $\varphi^{\,h}\eta_t$, est connu à l'instant t : c'est la prévision. L'erreur est donc entièrement portée par la somme, qui rassemble les aléas survenant entre t et $t + h$.

Ces aléas forment un bruit blanc : ils sont centrés, de même variance σ^2, et non corrélés entre eux. La variance d'une somme de termes non corrélés est la somme des variances, et chaque terme est multiplié par un coefficient qui sort au carré :

$$\mathrm{Var}\left(\sum_{j=0}^{h-1}\varphi^{\,j}\varepsilon_{t+h-j}\right) = \sum_{j=0}^{h-1}\varphi^{\,2j}\,\mathrm{Var}\big(\varepsilon_{t+h-j}\big) = \sigma^2\sum_{j=0}^{h-1}\big(\varphi^2\big)^j.$$

La dernière somme est géométrique, de raison φ^2. Le processus étant stationnaire, $|\varphi| < 1$ donc $\varphi^2 \neq 1$, et

$$\sum_{j=0}^{h-1} \left(\varphi^2\right)^j = \frac{1-\varphi^{2h}}{1-\varphi^2}.$$

D'où le résultat annoncé en section 3 :

$$\boxed{\mathrm{Var}\big(\eta_{t+h} - \hat{\eta}_{t+h}\big) = \sigma^2 \, \frac{1-\varphi^{2h}}{1-\varphi^2}}$$

H.3 Le seuil d'acceptation d'un paramètre par l'AIC

On part de la forme gaussienne du critère, à une constante près qui ne dépend pas du modèle :

$$\mathrm{AIC} = n \ln \hat{\sigma}^2 + 2k + \text{constante}.$$

Soient deux modèles ajustés sur les mêmes n observations, l'un à k paramètres et de variance résiduelle $\hat{\sigma}_k^2$, l'autre à $k+1$ paramètres et de variance $\hat{\sigma}_{k+1}^2$. On suppose le premier emboîté dans le second, c'est-à-dire que le modèle riche contient le modèle simple comme cas particulier. C'est ce qui se produit lorsqu'on passe de l'ordre p à l'ordre $p+1$: le modèle simple s'obtient en annulant le coefficient ajouté. Cette hypothèse garantit

$$\hat{\sigma}_{k+1}^2 \leq \hat{\sigma}_k^2,$$

puisque le modèle riche peut toujours reproduire l'ajustement du modèle simple.

La différence des deux critères vaut

$$\mathrm{AIC}_{k+1} - \mathrm{AIC}_k = n\left(\ln \hat{\sigma}_{k+1}^2 - \ln \hat{\sigma}_k^2\right) + 2 = n \ln \frac{\hat{\sigma}_{k+1}^2}{\hat{\sigma}_k^2} + 2.$$

Le critère est plus faible pour le modèle riche lorsque cette différence est négative :

$$n \ln \frac{\hat{\sigma}_{k+1}^2}{\hat{\sigma}_k^2} < -2 \qquad \Longleftrightarrow \qquad \frac{\hat{\sigma}_{k+1}^2}{\hat{\sigma}_k^2} < e^{-2/n}.$$

La baisse relative de variance résiduelle nécessaire est donc

$$1 - \frac{\hat{\sigma}_{k+1}^2}{\hat{\sigma}_k^2} \;>\; 1 - e^{-2/n} \;\simeq\; \frac{2}{n},$$

cf développement $e^{-x} \simeq 1 - x$ pour x petit.

Conséquence. Le seuil se comporte comme $2/n$ et tend vers zéro quand la série s'allonge. Sur une série longue, l'AIC finit par accepter presque tout paramètre supplémentaire. C'est ce qui explique la décroissance monotone observée sur le balayage d'ordres présenté dans les résultats.

H.4 Le biais de transfert d'une régression sur retards

On dispose d'une régression sur w retards, ajustée sur le domaine source :

$$\hat{y} = \sum_{j=1}^{w} a_j \, x_j + b.$$

L'ordonnée à l'origine

L'ajustement par moindres carrés avec ordonnée à l'origine impose que la somme des résidus soit nulle sur les données d'apprentissage. La moyenne des prédictions y égale donc la moyenne des cibles, ce qui donne

$$b = \bar{y} - \sum_{j=1}^{w} a_j \, \bar{x}_j.$$

Sur une série stationnaire, les retards et la cible ont la même moyenne, celle du domaine source, que je note $\mu_{\mathcal{S}}$. D'où

$$\boxed{b = \mu_{\mathcal{S}}\Big(1 - \sum_{j=1}^{w} a_j\Big)}$$

L'ordonnée à l'origine n'est donc pas un paramètre libre : elle mémorise la moyenne du domaine d'apprentissage, pondérée par ce que les coefficients ne restituent pas.

Le biais sur un autre domaine

On applique ce modèle à un domaine cible de moyenne $\mu_{\mathcal{T}} = \mu_{\mathcal{S}} + \Delta$. L'espérance de la prédiction y vaut

$$\mathbb{E}_{\mathcal{T}}[\hat{y}] = \sum_{j=1}^{w} a_j \, \mu_{\mathcal{T}} + b = \Big(\sum_{j=1}^{w} a_j\Big)\mu_{\mathcal{T}} + \mu_{\mathcal{S}}\Big(1 - \sum_{j=1}^{w} a_j\Big).$$

Le biais est l'écart à la vraie moyenne $\mu_{\mathcal{T}}$:

$$\mathbb{E}_{\mathcal{T}}[\hat{y}] - \mu_{\mathcal{T}} = \Big(1 - \sum_{j=1}^{w} a_j\Big)(\mu_{\mathcal{S}} - \mu_{\mathcal{T}})$$

soit

$$\boxed{\text{biais} = -\Big(1 - \sum_{j=1}^{w} a_j\Big)\,\Delta}$$

Lecture

Deux facteurs seulement interviennent, et ils se multiplient. Si $\sum_j a_j = 1$, le biais est nul quel que soit l'écart entre les domaines : un décalage constant de l'entrée se retrouve intégralement dans la sortie. Si les domaines ont la même moyenne, le biais est nul quelle que soit la somme des coefficients.

Vérification numérique

Sur la ville cible, la moyenne d'apprentissage du domaine source vaut 13,869 et la fraction non restituée $1 - 0{,}8823 = 0{,}1177$. La première formule prédit donc $b = 1{,}632$, à comparer à l'ordonnée à l'origine réellement ajustée, 1,6303. Avec $\Delta = 11{,}00$, la seconde prédit un biais de $-1{,}294$, à comparer au biais mesuré sur le test, $-1{,}370$.

Table des figures

1 L'espace de travail partagé du C19, où s'est déroulé le stage. 6
2 Les trois formes de changement de distribution. En trait plein le domaine source, en pointillés le domaine cible. 7
3 Les 5 décalages fabriqués sur données synthétiques. Trait plein bleu : domaine source. Pointillés rouges : domaine cible. 8
4 Autocorrélation de la température horaire après retrait de la tendance. Les pics à 24, 48 et 72 heures révèlent le cycle journalier. 11
5 Le découpage en fenêtres glissantes. Le modèle lit les 24 valeurs en bleu et prédit la valeur en orange, située 24 pas après la fin de la fenêtre. Les valeurs grises sont le futur au moment de la prédiction : elles ne sont pas lues. 16
6 Les 3 familles de méthodes d'adaptation, situées par l'endroit où elles interviennent dans la chaîne. 17
7 Exemple de paire source-cible générée pour les 5 décalages. 20
8 Le signal de la ville cible (Miami) : température brute, zoom sur quelques jours, saisonnalité retirée, et série désaisonnalisée obtenue. 21
9 Le prolongement de la saisonnalité. Les 4 blocs d'un an prélevés à la fin de l'apprentissage sont moyennés position par position, et le profil obtenu est répété sur le test. 22
10 Température brute pour les 4 villes source et la ville cible. 23
11 Le découpage du signal dans UCI HAR. 24
12 La structure de `X_train.txt` et son alignement avec les autres fichiers. La ligne i décrit partout la même fenêtre : ses 128 mesures brutes dans `Inertial Signals`, ses 561 statistiques dans `X_train.txt`, son activité dans `y_train.txt` et la personne qui l'a produite dans `subject_train.txt`. 25
13 L'ordre des lignes du fichier. En haut, les 7352 lignes d'apprentissage sont regroupées par sujet, chacun occupant un bloc contigu. Au milieu, les 341 lignes du sujet 3 se répartissent en 12 blocs d'activité, les 6 activités du protocole étant parcourues deux fois dans le même ordre. En bas, un bloc se décompose lui-même en tronçons, séparés par des coupures où le recouvrement de 50 % est interrompu. . 26
14 Signaux bruts pour deux activités comparés entre un sujet difficile à classer et un sujet facile. 27
15 Décomposition MSTL d'une série à deux périodes, 24 et 168 heures, détectées automatiquement par le périodogramme. 28
16 Comparaison des 9 méthodes sur le décalage de fréquence, aux 3 intensités. . . . 30
17 Erreur et biais des 11 ordres testés. 30
18 Transfert d'un modèle appris sur un bruit de mémoire $\varphi = 0{,}9$ vers des cibles de mémoire décroissante. 32
19 R^2 par ville. D'abord la température brute, ensuite la série désaisonnalisée. À gauche un modèle unique appris sur les 4 villes source, à droite un modèle par ville. 33
20 Écart-type de la série désaisonnalisée. Le changement de niveau à la frontière apprentissage-test mesure le défaut d'extrapolation de la saisonnalité, non un changement réel de météo. 34
21 Comparaison des méthodes d'adaptation sur la ville cible, série désaisonnalisée. La ligne horizontale marque le modèle appris directement sur la cible. 35
22 Erreur de prévision à 24 pas et AIC en fonction de l'ordre autorégressif. 37
23 Rappel par sujet et par classe sur le domaine cible. Les cases sombres localisent les difficultés que le score global dissimule. 39
24 Matrice de confusion sur les 3 sujets cible de test, et non sur les 9, d'où des rappels différents de ceux du tableau précédent. 39

25 Performance sujet par sujet. À gauche, chaque sujet source évalué par les 20 autres. À droite, chaque sujet cible évalué par le modèle source complet. Les deux panneaux ne mesurent pas la même chose : le premier est une validation croisée, le second un transfert. 40
26 Distribution des performances obtenues en retirant deux sujets au hasard, comparée au retrait ciblé des deux sujets les plus difficiles. 41
27 Validation croisée interne au domaine cible, où chaque sujet est évalué par les 8 autres. Le classement obtenu ne survit pas au transfert : le sujet le plus faible ici gagne 0,207 sous le modèle source, et l'un des moins bien classés devient le mieux classé de tous, à 0,995. 41
28 Courbes d'apprentissage à domaines équilibrés, 6 sujets de chaque côté. 42
29 Apport de 100 fenêtres source quand la cible n'en fournit que quelques-unes. . . . 42
30 Performance en fonction du rappel vers la source, pour un modèle source fort et un modèle source faible. 43